

Brain Tumor Classification Using Deep Learning with a Web-Based Interface

**Hamza Nawabuddin
BSc Computer Science
Final Year Project**

**April 2026
University of Salford**

Supervisor: Dr. Ahmed Al-Adaileh

Student Name	Hamza Nawabuddin
Student ID Number	00709218
Level	6
Programme of Study	Computer Science
Module Title	Final Year Project
Lecturer	Dr. Rowani Odum
Supervisor	Dr. Ahmed Al-Adaileh
Assessment Task/Title	Dissertation
Assessment Due Date	April 23rd, 2026
Word Count (if applicable)	10,209 words
Declaration (Please e-sign your name) <i>I declare that:</i> <i>This work is my own.</i> <i>If this is a group project, each student has contributed to the work in accordance with the set criteria.</i> <i>The work of others used in its completion has been duly acknowledged.</i> <i>Experimental or other investigative results have not been falsified.</i> <i>I have read and understood the Academic Misconduct Procedure.</i>	Hamza Nawabuddin

Abstract

This dissertation aims to develop a model capable of classifying brain tumors using deep learning techniques which could further assist radiologists in identifying brain tumors, like glioma, meningioma, pituitary and no tumor. In addition, a web-based application was developed to allow users upload MRI images and receive predictions.

A transfer learning approach was used during training to evaluate between three Convolutional Neural Networks (CNNs), including VGG16, ResNet50, and EfficientNetB0. The models were trained on a publicly available MRI dataset with over 7,000 MRI images. The models were applied with techniques such as preprocessing, data augmentation, and fine-tuning to improve performances. ResNet50 showed the best results, reaching a test accuracy of 99.0%, and was selected for integration into the Flask web application.

The model was evaluated using confusion matrices, which showed strong performance across all classes, with only minor misclassifications between tumour types such as glioma and meningioma. Overall, the results suggest that deep learning can be effective for brain tumor classification. However, there are still some limitations, such as relying on a single dataset and not including tumour localisation, meaning further work is needed before real-world use.

Acknowledgements

I wouldn't have been able to complete this dissertation without the support of my supervisor, **Dr. Ahmed Al-Adaileh**. I truly appreciate the guidance he has provided throughout my academic year, not only as a supervisor but also as a lecturer. His way of teaching is unmatched, and whenever I needed reassurance, he would always provide me with the feedback I needed and encourage me to push myself further. I am truly grateful for the amount of time, effort and support he has given me.

Lastly, I would like to thank my family and friends for supporting me and for always encouraging me to believe in myself. Thank you to my dad, sisters, aunt, nephew and my best friends. I will always keep them close to my heart.

Dedication

This work is dedicated to my late mother, who bravely battled a brain tumour. Although our time together was limited, I know she would have been proud to see me graduate. I chose this topic to feel closer to her.

Table of contents

[Abstract](#)

[Acknowledgements](#)

[Dedication](#)

[List of Figures](#)

[List of Tables](#)

[List of Abbreviations](#)

[Chapter 1 - Introduction](#)

[1.1 Motivation for the project](#)

[1.2 Aim](#)

[1.3 Objectives](#)

[1.3 Approach Adopted](#)

[1.4 Summary of the report](#)

[1.5 Gantt Chart](#)

[Figure 1.5 - Gantt Chart illustrating the timeline of the entire project](#)

[Chapter 2 - Literature Review](#)

[2.1. Introduction](#)

[2.2 Review of existing literature](#)

[2.3 Literature Review Conclusion](#)

[Chapter 3 - Methodology](#)

[3.1 Project Management Methodology](#)

[3.2 Development methodology](#)

[3.3 Machine learning pipeline](#)

[3.4 Algorithms](#)

[3.5 Architectures](#)

[3.6 Tools, Techniques, Programming Languages, Frameworks, and Libraries](#)

[3.7 Datasets](#)

[3.8 Evaluation techniques](#)

[3.9 System design overview](#)

[3.10 Methodology Conclusion](#)

[Chapter 4 - Requirements Specification, Design, and Planning](#)

[4.1 Introduction](#)

[4.2 System Scope and Stakeholders](#)

[4.2.1 System Scope](#)

[4.2.2 Primary Stakeholders](#)

[4.2.2.1 Medical Professionals \(Doctors\)](#)

[4.2.3 Secondary Stakeholders](#)

[4.2.3.1 Developer](#)

[4.2.3.2 Healthcare Institutions](#)

[4.3 Requirements Specification](#)

[4.3.1 Functional Requirements](#)

[4.3.1.1 User Interaction and Navigation](#)

[4.3.1.2 MRI Image Processing and Prediction](#)

[4.3.2 Non-Functional Requirements](#)

[4.3.2.1 Performance](#)

[4.3.2.2 Usability](#)

[4.3.2.3 Reliability](#)

[4.3.2.4 Accuracy](#)

[4.4 Development Approach](#)

[Table 4.4 - Development Stages of the system](#)

[4.5 System Interaction and Workflow Design](#)

[4.5.1 Use cases](#)

[4.5.2 Use Case Diagram](#)

[Figure 4.5.3 - Use case diagram of the brain tumor detection system.](#)

[4.5.3 Textual use case](#)

[Table 4.5.4 - Textual use case describing the tumor detection process.](#)

[4.5.5 Textual HTA](#)

[4.5.6 Sequence Diagram](#)

[Figure 4.5.6 - Sequence diagram of the brain tumour detection process](#)

[4.6 High-Level System Architecture](#)

[Figure 4.6 - High-level deep learning model architecture \(Author's own work, created using Flowchart, images sourced from Freepik and Khandekar, 2023 \)](#)

[4.7 Model Design](#)

[4.7.1 Dataset Preparation](#)

[4.7.2 Preprocessing and Augmentation](#)

[Figure 4.7.2 - Example of MRI images before and after preprocessing through resizing to a fixed dimension of 512 × 512 pixels](#)

[4.7.3 Model Architecture](#)

[Figure 4.7.3 Proposed ResNet50 CNN model for brain tumor classification \(Author's own work, created using Figma\)](#)

[4.8 Web Application Design](#)

[4.8.1 Splash Screen](#)

[Figure 4.8.1 - Wireframe for Splash Screen](#)

[Figure 4.8.2 - Final Splash Screen Interface](#)

[4.8.2 Language Selection Screen](#)

[Figure 4.8.3 - Wireframe for Language Screen](#)

[Figure 4.8.4 - Final Language Screen Interface](#)

[4.8.3 Main Menu](#)

[Figure 4.8.5 - Wireframe for Main Menu Screen](#)

[Figure 4.8.6 Final Main Menu Interface](#)

[4.8.4 Prediction Screen](#)

[Figure 4.8.7 - Wireframe for Prediction Screen](#)

[Figure 4.8.8 - Final Prediction Screen Interface](#)

[4.8.5 Result Screen](#)

[Figure 4.8.9 - Wireframe for Results Screen](#)

[Figure 4.8.10 - Final Results Screen Interface with Arabic](#)

[4.9 Chapter Summary](#)

[Chapter 5 - Development and Implementation](#)

[5.1 Model development and training](#)

[5.1.1 Dataset preparation](#)

[Figure 5.1.1 - MRI images from the dataset across four tumor classes \(glioma, meningioma, nontumor and pituitary\)](#)

[5.1.2 Model exploration](#)

[Figure 5.1.2 - Model summary of ResNet50 with parameters](#)

[Figure 5.1.2. Model summary for EfficientNetB0 with parameters](#)

[Figure 5.1.2 - Model summary for VGG16 with parameters](#)

[5.1.3 Final model implementation](#)

[5.1.4 Training configuration](#)

[5.1.5 Model saving and export](#)

[5.2 Flask web application development](#)

[5.2.1 Project structure](#)

[5.2.2 Additional features](#)

[5.3 Web Application Interface](#)

[5.4 Challenges and Solutions](#)

[Figure 5.4.1 Phase 1, 2 and without resizing the dataset equally \(VGG16\)](#)

[Figure 5.4.2 Removal of phase 1 and 2 and resizing the dataset equally. \(VGG16\)](#)

[5.5 Chapter summary](#)

[Chapter 6 - Testing and Results](#)

[6.1 Experimental Setup](#)

[6.2 Training and Validation Results](#)

[6.2.1 VGG-16](#)

[Figure 6.2.1 VGG16 training and validation \(Accuracy and Loss\)](#)

[6.2.2 ResNet50](#)

[Figure 6.2.2 ResNet50 training and validation \(Accuracy and Loss\)](#)

[6.2.3 EfficientNetB0](#)

[Figure 6.2.3 EfficientNetB0 training and validation \(Accuracy and Loss\)](#)

[6.3 Testing Results](#)

[Figure 6.3 Comparison of testing results between the models](#)

[6.4 System and Input Testing](#)

[6.4.1 Invalid Input Testing](#)

[Figure 6.4.1 - Invalid input testing for images of a dog and house](#)

[6.5 Front-End Validation](#)

[Figure 6.5.1 HTML validation for language.html](#)

[Figure 6.5.1 CSS validation for language.html](#)

[6.6 Confusion Matrix and Classification Analysis](#)

[Figure 6.6 - Confusion Matrix for ResNet50](#)

[6.7 Chapter Summary](#)

[Chapter 7 - Critical Evaluation](#)

[7.1 Achievement of objectives](#)

[Objective 1: Conduct a literature review on brain tumor detection and find suitable deep learning approaches](#)

[Objective 2: Identify and prepare a suitable dataset](#)

[Objective 3: Find and compare different deep learning models](#)

[Objective 4: Develop a deep learning model and achieve an accuracy above 95%](#)

[Objective 5: Develop a user interface for the system](#)

[Objective 6: Additional features for the system](#)

[Objective 7: Testing of the final system](#)

[7.2 Strengths of the project](#)

[7.3 Limitations and weaknesses](#)

[7.4 Deviations from Project Plan](#)

[7.5 Ethical, legal, and practical concerns](#)

[7.6 Lessons learned](#)

[7.8 Reflection on the first two dissertation deliverables](#)

[7.9 Chapter summary](#)

[Chapter 8 - Conclusions](#)

[8.1 Overall summary](#)

[8.2 Key Findings](#)

[8.3 Future work](#)

[8.4 Comparison with State-of-Art results](#)

[Table 8.4 - Comparison of research results with the same dataset](#)

[8.5 Professional and Ethical Considerations](#)

[Reference list](#)

[Appendix A: Project Proposal](#)

[1. Introduction](#)

[1.2 Background Study](#)

[1.3 Beneficiaries](#)

[1.4 Personal Motivation](#)

[2. Objectives](#)

[2.1 Gantt Chart](#)

[Figure 2: Gantt chart](#)

[3. Methodology](#)

[3.1 Software requirements](#)

[3.2 Hardware requirements](#)

[3.3 Development Methodology for the project](#)

[Figure 3: Iterative model](#)

[Appendix B: Project Logbook](#)

List of Figures

[Figure 1.5 - Gantt Chart illustrating the timeline of the entire project](#)

[Figure 4.5.3 - Use case diagram of the brain tumor detection system](#)

[Figure 4.5.6 - Sequence diagram of the brain tumour detection process](#)

[Figure 4.6 - High-level deep learning model architecture](#)

[Figure 4.7.2 - Example of MRI images before and after preprocessing through resizing to a fixed dimension of \$512 \times 512\$ pixels](#)

[Figure 4.7.3 Proposed ResNet50 CNN model for brain tumor classification](#)

[Figure 4.8.1 - Wireframe for Splash Screen](#)

[Figure 4.8.2 - Final Splash Screen Interface](#)

[Figure 4.8.3 - Wireframe for Language Screen](#)

[Figure 4.8.4 - Final Language Screen Interface](#)

[Figure 4.8.5 - Wireframe for Main Menu Screen](#)

[Figure 4.8.6 Final Main Menu Interface](#)

[Figure 4.8.7 - Wireframe for Prediction Screen](#)

[Figure 4.8.8 - Final Prediction Screen Interface](#)

[Figure 4.8.9 - Wireframe for Results Screen](#)

[Figure 4.8.10 - Final Results Screen Interface with Arabic](#)

[Figure 5.1.1 - MRI images from the dataset across four tumor classes \(glioma, meningioma, nontumor and pituitary\)](#)

[Figure 5.1.2 - Model summary of ResNet50 with parameters](#)

[Figure 5.1.2. Model summary for VGG16 with parameters](#)

[Figure 5.1.2 - Model summary for EfficientNetB0 with parameters](#)

[Figure 5.4.1 Phase 1, 2 and without resizing the dataset equally \(VGG16\)](#)

[Figure 5.4.2 Removal of phase 1 and 2 and resizing the dataset equally. \(VGG16\)](#)

[Figure 6.2.1 VGG16 training and validation \(Accuracy and Loss\)](#)

[Figure 6.2.2 ResNet50 training and validation \(Accuracy and Loss\)](#)

[Figure 6.2.3 EfficientNetB0 training and validation \(Accuracy and Loss\)](#)

[Figure 6.3 Comparison of testing results between the models](#)

[Figure 6.5.1 HTML validation for language.html](#)

[Figure 6.5.1 CSS validation for language.html](#)

[Figure 6.6 - Confusion Matrix for ResNet50](#)

[Figure 7.4.1 Phase 1, 2 and without resizing the dataset equally \(VGG-16\)](#)

[Figure 2: Gantt chart](#)

[Figure 3: Iterative model](#)

List of Tables

[Table 4.4 - Development Stages of the system](#)

[Table 4.5.4 - Textual use case describing the tumor detection process.](#)

[Table 8.4 - Comparison of research results with the same dataset](#)

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
CNN	Convolutional Neural Network
MRI	Magnetic Resonance Imaging
VGG16	Visual Geometry Group 16
ResNet50	Residual Network 50
EfficientNetB0	EfficientNet Model
API	Application Programming Interface
UI	User Interface
RGB	Red, Green, Blue

Chapter 1 - Introduction

1.1 Motivation for the project

Brain tumor detection is an important task, as identifying tumors earlier can support diagnosis and improve outcomes for patients (K.V & King, 2023). However, analyzing MRI scans is a complex and time-consuming process which is heavily dependent on radiologists. This project is motivated by developing a system that can assist in detecting MRI images and provide faster and more consistent predictions. The goal is not to replace medical professionals, but to support them by suggesting them with a tool that can improve their diagnosis process and efficiency.

1.2 Aim

The aim of this project is to build a model designed to identify brain tumors, like glioma, meningioma, pituitary and non tumor classes, while also providing a user interface (UI) to upload different MRI images and receive predictions.

1.3 Objectives

1. To review existing research on brain tumor detection using deep learning
2. To review a suitable brain tumor MRI dataset
3. To experiment using multiple deep learning models with transfer learning such as, VGG16, ResNet50 and EfficientNetB0
4. To evaluate and select the models performance with fine-tuning
5. To design and develop a web-based application in Flask
6. To integrate the test model into Flask for predication
7. To test and evaluate the system

1.3 Approach Adopted

The project was developed using an iterative approach, allowing the system to be improved and modified through different approaches. Multiple deep learning models were taken into consideration using transfer learning techniques. The final model was selected based on its accuracy, and then was integrated into a Flask application. These iterations were done through different stages ensuring the model and the application were developed together.

1.4 Summary of the report

The report is structured as follows :

- Chapter 2: Literature review presents existing approaches to brain tumor detection using deep learning and key limitations
- Chapter 3: Methodology outlines the development approach used in this project, such as machine learning pipeline and tools used.
- Chapter 4: Requirements Specification, Design, and Planning describes the overall designing and planning of the system.
- Chapter 5: Development and Implementation discusses the development both for the deep learning model and Flask application
- Chapter 6: Testing and Results presents the models performance
- Chapter 7: Critical Evaluation discusses the strengths, limitations and challenges gathered
- Chapter 8: Conclusion concludes the report by summarizing key findings, comparison with state-of-art and suggestions on improvements for future work

1.5 Gantt Chart

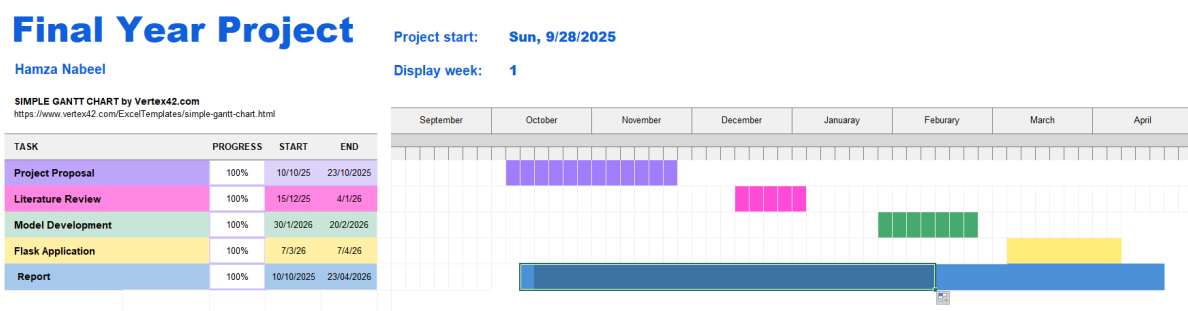


Figure 1.5 - Gantt Chart illustrating the timeline of the entire project

Chapter 2 - Literature Review

2.1. Introduction

In healthcare, machine learning has the potential to transform patient outcomes, enhance the quality of life, and more importantly make healthcare more accessible and cost-effective. Traditionally, disease diagnosis relies on the experience of professionals which is time consuming. Currently, machine learning can analyze huge amounts of data faster and identify subtle patterns far more accurately than humans. There are lots of benefits for machine learning in health care such as being able to identify diagnosis earlier, reduction of health costs, and improving towards patient care. (EIT health, 2024).

This section presents a review of existing literature related to the aim of my project. The review examines state-of-the-art approaches, including convolutional neural networks, transfer learning models, and classification systems.

The findings from this literature review are used to identify existing research gaps and limitations that motivate the proposed approach. These insights directly inform the design choices, tools, and techniques discussed in the methodology chapter that follows

2.2 Review of existing literature

Paper 1 - Brain Tumor Detection Based on Deep Learning Approaches and Magnetic Resonance Imaging

In 2023, Abdusalomov et al., proposed a state of the art brain tumor detection framework which is based on a fine-tuned YOLOv7 model. The main objective of the research was to develop an automated system that would be able to accurately detect and localize brain tumors in MRI images. A large publicly available dataset was used to improve robustness, while data augmentation techniques were applied to enhance the image's quality and to address limitations associated with low-resolution MRI scans.

The researchers integrated the Convolutional Block Attention Module (CBAM), enabling the network to focus on spatially and channel-wise relevant tumor features, which improved the detection performance. Additionally, architectural enhancements such as Spatial Pyramid Pooling Fast Plus (SPPF+) and Bi-directional Feature Pyramid Network (BiFPN) were introduced to improve multi-scale feature fusion, particularly for detecting small tumors.

Despite these strong results, there were several limitations such as the detection of very small tumors remains challenging, even with multi-scale feature fusion mechanisms. Furthermore, the use of bounding-box detection may result in non-specific tumor boundaries compared to segmentation-based approaches. The model also relies heavily on large datasets, making it both costly and time-consuming. To conclude, the study lacks validation

on real data, which limits the immediate applicability of the proposed system in medical practice.

Paper 2 - MRI-based brain tumour image detection using CNN based deep learning method

In 2022, Chattopadhyay and Maitra proposed a CNN-based deep learning method for automatic brain tumour detection using MRI images. The aim of their study was to reduce the needs of manual analysis by medical professionals and to improve efficiency. Their proposed CNN model achieved a high accuracy of 99.74%, demonstrating the effectiveness of deep learning for tumour classification.

The authors discussed three segmentation approaches: manual, semi-automatic, and fully automatic methods. Manual segmentation was identified as time-consuming and dependent on radiologists, while semi-automatic methods reduced the need of a professional but still introduced user variability. As a result, the study focused on a fully automatic CNN-based approach to improve accuracy and reliability.

The model was evaluated using the BraTS 2020 dataset, consisting of 2,892 MRI images from different modalities. The best performance was achieved using a Softmax activation function and RMSProp optimizer. However, the study is limited to binary classification, which lacks precise tumour localization and relies on a relatively small dataset. Additionally, it was not validated on real medical data, which may affect generalizability of the system.

Paper 3 - ResNet50-based Deep Learning model for accurate brain tumor detection in MRI scans

In 2024, Rath et al. proposed a deep learning model based on the ResNet50 architecture for automating brain tumor detection with MRI images. In their research, the dataset consisted of 2,577 images, containing both tumor and healthy brains. To improve the model's generalization and accuracy readings, extensive data augmentation techniques were applied during training.

The model was developed using transfer learning, using ResNet50 which is a pre-trained model, and it was fine-tuned for a binary classification with MRI scans of brain tumors and healthy images. The evaluation was performed in a Google Colab environment, where both TensorFlow and Keras libraries were used. Moreover, their model achieved a test accuracy of 97.35%, with good precision, recall, and F1-scores. However, the researchers do acknowledge several limitations being presented in their work, including the relatively small dataset size, demand of high computational requirements when it's associated with deep architecture, and lastly the need for further validation across diverse data to ensure robustness and real-world applicability.

Paper 4 - Enhancing Brain Tumor Detection Using a CNN-Powered Android Application

In 2025, Zamil and Lee proposed a solution for brain tumor detection using a convolutional neural network (CNN) integrated into an Android application. The objective of their research was to address the limitations of current diagnostic systems by enabling real-time, on-device brain tumor detection using mobile technology. The proposed system was classified into four categories of MRI images: glioma, meningioma, pituitary tumor, and non-tumorous. A publicly available MRI dataset from Kaggle which consisted of over 7,000 images was used. Google Colab's environment was used to perform model training, after which the model was converted into a TensorFlow Lite format to be used in the development of android devices. The application was developed using Android Studio IDE. The system achieved a training accuracy of 99.61%, with testing and validation accuracies of approximately 94– 95%, demonstrating strong classification performance.

Despite its results, several limitations were identified. The model was trained and evaluated using a single public dataset. In addition, the study did not conduct extensive comparisons with alternative CNN architectures or optimization strategies. However, mobile deployment does improve usability, the system still lacks from using a single dataset, which is necessary before being used by professionals.

2.3 Literature Review Conclusion

This literature review examined a range of deep learning-based approaches for brain tumor detection using MRI scans, including detection, classification, transfer learning, model comparison, and mobile deployment strategies. The study demonstrates that convolutional neural networks and transfer learning architectures can achieve high accuracy in identifying brain tumors. However, with limitations, including challenges in detecting small tumors, limited localization and datasets. Moreover, recent efforts toward mobile deployment highlights promising directions for improving accessibility, where further research is required to ensure robustness and clinical applicability. These findings indicate a clear research gap for developing an efficient, accurate, and deployable brain tumor detection system that addresses both technical and practical limitations. The next chapter presents the methodology adopted to design and implement a system that directly responds to these identified gaps.

Chapter 3 - Methodology

3.1 Project Management Methodology

Waterfall model -

The waterfall model is a traditional, linear software development methodology in which development progresses sequentially through fixed phases such as requirements, design, implementation, testing, demand maintenance. Each phase must be completed before moving to the next. Despite this approach being effective for projects with stable and well defined requirements, it lacks flexibility and does not support continuous feedback or iteration. As a result, it is less suitable for machine learning projects, where models are required to repeat experiments, tuning, and refinement. (GeeksforGeeks, 2025).

Iterative model -

The Iterative model is a software development methodology in which the system is developed through repeated cycles, with each iteration producing an improved version of the product. Development starts with an initial design based on the requirements, which is then implemented, tested, and refined across multiple iterations. This approach supports early validation and regular feedback. (tutorialspoint, n.d.).

Based on this evaluation, the Iterative model was selected for this project as it aligns with the data driven nature of machine learning development. The model supports repeated training, evaluation, and refinement of the brain tumour detection system, enabling performance improvements to be made throughout the project.

3.2 Development methodology

This section describes the technical methodology adopted to design and implement the brain tumour detection system. The selected approach is informed directly by the reviewed literature, particularly studies focusing on CNN-based MRI classification, transfer learning architectures, and mobile deployment of deep learning models.

3.3 Machine learning pipeline

Based on common practices observed in the reviewed studies, the system follows a standard machine learning pipeline. Initially, MRI images are obtained from publicly available datasets, primarily sourced from Kaggle, as used in the mobile CNN study by Zamil and Lee (2025). The images are preprocessed through resizing and normalization to ensure consistency across inputs. Data augmentation techniques are applied to improve generalization.

The dataset is then split into training, validation, and testing sets, consistent with methodologies used in Papers 2 and 3. Model training is performed iteratively, with validation results guiding adjustments to model architecture and training parameters. Final evaluation is conducted using test data.

3.4 Algorithms

Convolutional Neural Networks (CNNs) are used as the primary algorithm for this project, reflecting their widespread adoption across all reviewed papers. CNNs are particularly effective for MRI image analysis due to their ability to automatically extract spatial features such as edges, textures, and tumour-related patterns. (Abdusalomov et al., 2023).

The literature demonstrates successful use of both custom CNN models and transfer learning approaches such as ResNet50. These findings inform the use of a CNN-based approach that balances performance and efficiency. (Rath et al., 2024; Zamil & Lee, 2025).

3.5 Architectures

The architectural design of the system is influenced by both deep and lightweight CNN architectures reviewed in the literature. ResNet50-based models (Rath et al., 2024) demonstrate strong classification performance using transfer learning, while mobile-focused architectures (Zamil and Lee, 2025) show that simpler CNN designs can achieve reliable results with reduced computational requirements.

3.6 Tools, Techniques, Programming Languages, Frameworks, and Libraries

Python is used as the primary programming language for the model's development due to its extensive support for machine learning. TensorFlow and Keras are employed for building and training the CNN models.

Image preprocessing is performed using OpenCV, while NumPy supports numerical operations and Matplotlib is used for visualizing training and evaluation results. Lastly, the model training and experimentation are conducted using Google Colab, allowing access to GPU acceleration, as adopted in multiple studies.

For mobile-related considerations referenced in the literature, Kotlin is identified as the programming language used for Android application development, particularly for deploying trained models using TensorFlow Lite. (Abdusalomov et al., 2023; Rath et al., 2024; Zamil & Lee, 2025).

3.7 Datasets

The dataset used in this project is the Brain Tumor MRI Dataset obtained from Kaggle. The dataset contains over 7,023 labelled MRI images across four categories: glioma, meningioma, pituitary tumour, and no tumour. The dataset is compiled from multiple publicly available sources, including Figshare, the SARTAJ dataset, and the Br35H dataset, and is released under a CC0 public domain license. (Nickparvar, 2021).

3.8 Evaluation techniques

The model's performance is assessed using accuracy and loss metrics during training and validation. Visualization tools such as confusion matrices are used to analyse classification outcomes, while precision, recall, and F1-score provide deeper insight into the model's ability to correctly identify tumour cases. The use of multiple metrics addresses limitations highlighted in the literature, where reliance on accuracy alone may be misleading in medical diagnosis tasks. (Chattopadhyay & Maitra, 2022; Rath et al., 2024).

3.9 System design overview

The proposed system follows a sequential design in which MRI images are first pre-processed and then passed through a CNN-based model for feature extraction and classification. The trained model outputs a predicted class indicating tumour type or absence of a tumour. Evaluation metrics are then applied to assess system performance. This design aligns with CNN-based brain tumour detection systems reported in the literature. (Abdusalomov et al., 2023; Rath et al., 2024; Zamil & Lee, 2025).

3.10 Methodology Conclusion

In this project, an Iterative model is used to manage the development of the system, allowing the brain tumour detection system to be trained, tested, and improved over multiple iterations. The system follows a standard machine learning pipeline, where MRI images are collected from Kaggle, preprocessed through resizing and normalization, and enhanced using data augmentation, dropout, regularization to prevent from overfitting. The dataset is split into training, validation, and testing sets, with model performance improved through iterative training. The system is developed using Python, with TensorFlow and Keras for model training, OpenCV for image preprocessing, and NumPy and Matplotlib for analysis. Transfer learning architectures such as ResNet50 is used for MRI image classification. Model performance is evaluated using accuracy, loss, precision, recall, F1-score, and confusion matrices, providing a clear and reliable way to assess results. Once the model is completed with good results, it is converted using TensorFlow Lite to support deployment on mobile devices. The mobile application is developed in Android Studio using Kotlin, where the trained model is integrated for on-device inference.

Chapter 4 - Requirements Specification, Design, and Planning

4.1 Introduction

This chapter outlines the requirements and design of the proposed brain tumour detection system. It starts by identifying stakeholder needs and translating them into functional and non-functional requirements. The chapter then describes how the main components of the system, including a deep learning model and the Flask web application, work together. The system was developed using an iterative approach, allowing continuous testing and improvement throughout the project.

4.2 System Scope and Stakeholders

This section defines the scope of the system and identifies the key stakeholders. It outlines the intended users of the system.

4.2.1 System Scope

The system is a web-based application designed to assist in the early detection of brain tumours from MRI scans. Medical professionals can upload MRI scans, which are processed by a deep learning model to classify them into four categories: glioma, meningioma, pituitary, or no tumour.

The system focuses on providing a quick and accessible indication rather than a confirmed diagnosis. It includes basic features such as image upload, preprocessing, classification, and displaying results with a confidence score.

The accuracy of the system depends on the input data and training dataset. Therefore, results should be interpreted with caution and verified by medical professionals.

4.2.2 Primary Stakeholders

4.2.2.1 Medical Professionals (Doctors)

Medical professionals are the main users of the system. They can use it as a supporting tool when reviewing MRI scans. They require accurate and reliable predictions that can assist in decision-making, although the system is not intended to replace professional diagnosis.

4.2.3 Secondary Stakeholders

4.2.3.1 Developer

The developer is responsible for building and maintaining the system, including training the deep learning model and integrating it with the Flask application. They require the system to be stable, maintainable, and easy to update.

4.2.3.2 Healthcare Institutions

They may adopt the system to support diagnostic workflows and improve efficiency. They require the system to be reliable and suitable for clinical use.

4.3 Requirements Specification

This section describes the functional and non-functional requirements of the system. It describes the key features and operations that the application must perform to assist in tumor detection.

4.3.1 Functional Requirements

This subsection outlines the functional requirements of the system, describing the key features and operations that the application must perform to support user interaction and tumour prediction.

4.3.1.1 User Interaction and Navigation

The system shall allow medical professionals to:

- Access the application through a web browser
- Select their preferred language (English or Arabic)
- Navigate between the main interface and prediction page
- Return to the main screen after completing tasks

4.3.1.2 MRI Image Processing and Prediction

The system shall:

- Allow users to upload MRI scans
- Automatically preprocess images before analysis
- Classify the MRI scans into one of four categories (glioma, meningioma, pituitary tumour, or no tumour)
- Display the prediction result along with a confidence score
- Allow users to upload another image for further predictions

These functional requirements are designed to reflect the standard workflow used by radiologists when analyzing MRI scans.

4.3.2 Non-Functional Requirements

This subsection defines the non-functional requirements of the system, focusing on performance, usability, reliability and accuracy considerations which is the overall quality of the system.

4.3.2.1 Performance

The system shall return tumour prediction results within a few seconds under normal operating conditions, ensuring that it can support time-sensitive diagnostic workflows. This is important as delays in prediction could affect the efficiency of radiologists when analysing MRI scans.

4.3.2.2 Usability

The system shall provide a clear and intuitive interface that can be easily used by medical professionals with minimal training. The interface design should reduce complexity and allow users to quickly upload images and gather results without confusion.

4.3.2.3 Reliability

The system shall operate consistently without crashing during image upload. It is essential to ensure that users can trust the system without repeatedly using the system.

4.3.2.4 Accuracy

The system shall provide users with reliable classification results based on the trained deep learning model. Accuracy is critical in medical applications, as incorrect predictions may negatively influence decision-making and reduce confidence in the system.

4.4 Development Approach

This section describes the development approach adopted for the system, explaining how the methodology supports the use of an iterative model. This approach assisted both the development of the deep learning model and the Flask application.

The system was designed and implemented using an iterative model, which allows development to occur through repeated cycles of designing and development, followed by testing and implementation. This approach was selected due to the nature of deep learning models, where models performance improves over continuous adjustments. In each iteration, different models are explored and fine-tuned to determine the most suitable model.

Furthermore, the iterative approach will support the integration of the trained model into Flask application. By developing the model and designing the interface alongside each other, the system can be completed progressively faster. The overall development process is summarised in Table 4, which outlines the key stages followed throughout the project.

Stages	Task
Stage 1	Gather suitable datasets
Stage 2	Initial model development and training
Stage 3	Model evaluation and comparison
Stage 4	Fine-tuning and data augmentation
Stage 5	Integration of the model into Flask
Stage 6	System testing and validation

Table 4.4 - Development Stages of the system

4.5 System Interaction and Workflow Design

This section presents how users interact with the system and outlines the overall workflow of the application. It includes use cases and interaction models to illustrate how the system supports tumour prediction tasks.

4.5.1 Use cases

Use cases describe how users interact with the system to achieve specific tasks. They provide a user-centred view of system functionality and help illustrate the overall workflow of the application. In this system, the primary actor is the radiologist, who interacts with the application to upload MRI scans and receive tumour predictions.

4.5.2 Use Case Diagram

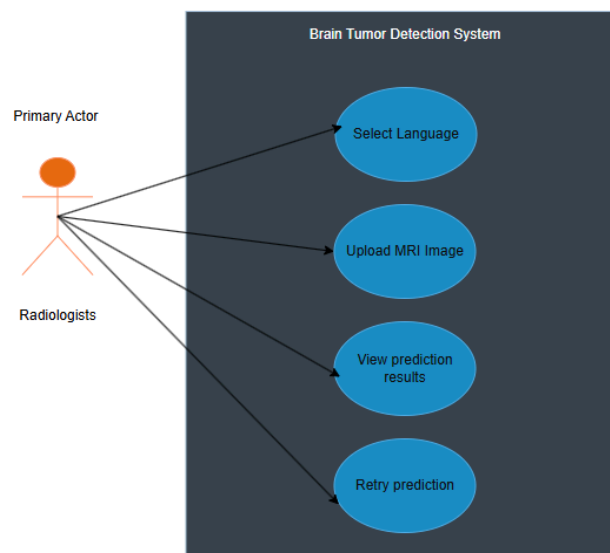


Figure 4.5.3 - Use case diagram of the brain tumor detection system.

4.5.3 Textual use case

Use Case Title	Tumor Detection
Primary Actor	Radiologist
Goal	To upload an MRI scan and receive a tumour prediction to support diagnosis
Scope	Brain tumor detection system
Preconditions	• The user is able to access the application • An MRI scan is available
Postconditions	The system displays the predicted tumour type and confidence score
Main Success Scenario	1. The radiologist opens the application 2. The radiologist selects their preferred language 3. The radiologist selects the prediction option 4. The radiologist uploads an MRI scan 5. The system preprocesses the MRI scan 6. The system generates a prediction 7. The system displays the tumour classification and confidence score 8. The radiologist reviews the result
Extension	4a) if the uploaded image is not a valid MRI scan, the system may produce inaccurate predictions.

Table 4.5.4 - Textual use case describing the tumor detection process.

4.5.5 Textual HTA

A Hierarchical Task Analysis (HTA) is used to break down the user interaction into structured steps, showing how tasks are completed within the system.

Plan 0: Do 1,2 3 then either option in 4

0: Use the Brain Tumor Detection Application

Plan 1: Do 1.1 then 1.2

1: Access the Application

1.1 Locate and open the application

Plan 1.2: Do 1.2.1 and 1.2.2

1.2 Launch the application

1.2.1 The app will display a splash screen

1.2.2 The screen then displays the language option screen

Plan 2: Do 2.1, 2.2 and then 2.3 in order

2: Choose your preferred language

2.1 Choose either English or Arabic

2.2 The system processes language selection

2.3 The screen will direct to the main menu

Plan 3: Do 3.1 or 3.2

3: Select an option in the main menu

3.1 Select "Predict Tumor"

3.2 Select to return to the language option screen

Plan 4 Do 4.1, 4.2, 4.3, 4.4 in order, and then choose either option for 4.5

4: Choose to predict a tumor

4.1 Select the upload option

4.2 Select an MRI scan

4.3 Select on predict (submit the image)

4.4 The system will display the prediction results

4.5 Choose the next action

4.5.1 Retry the prediction

4.5.2 Return to the main menu

4.5.6 Sequence Diagram

The sequence diagram illustrates the interaction between the user, the web interface, the Flask backend, and the deep learning model. It shows how an MRI image is uploaded, processed by the backend, passed to the model for prediction, and then returned to the user as a result.

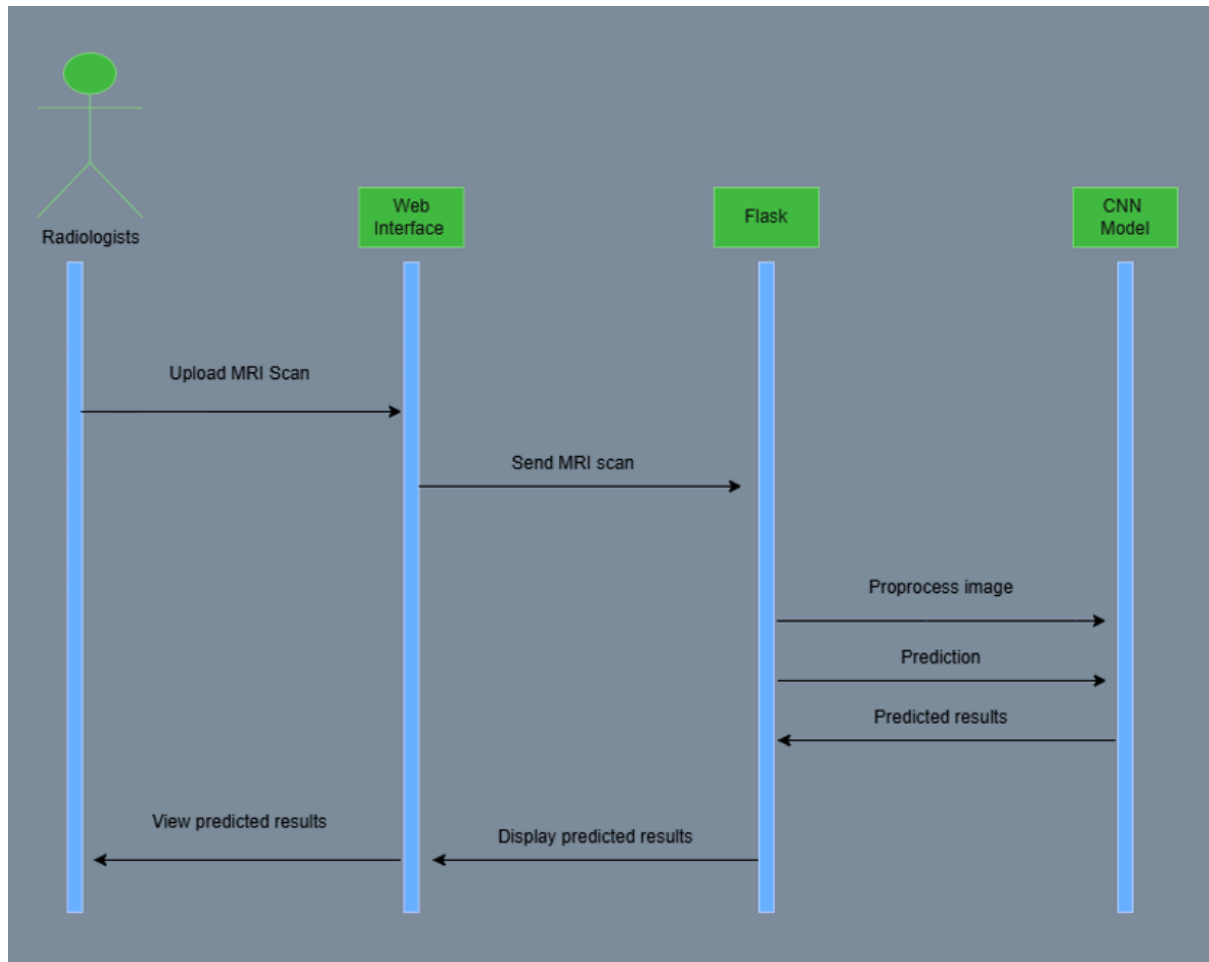


Figure 4.5.6 - Sequence diagram of the brain tumour detection process

4.6 High-Level System Architecture

This section provides an overview of the system architecture, illustrating how the front-end interface, backend processing, and deep learning model are integrated to perform tumour classification.

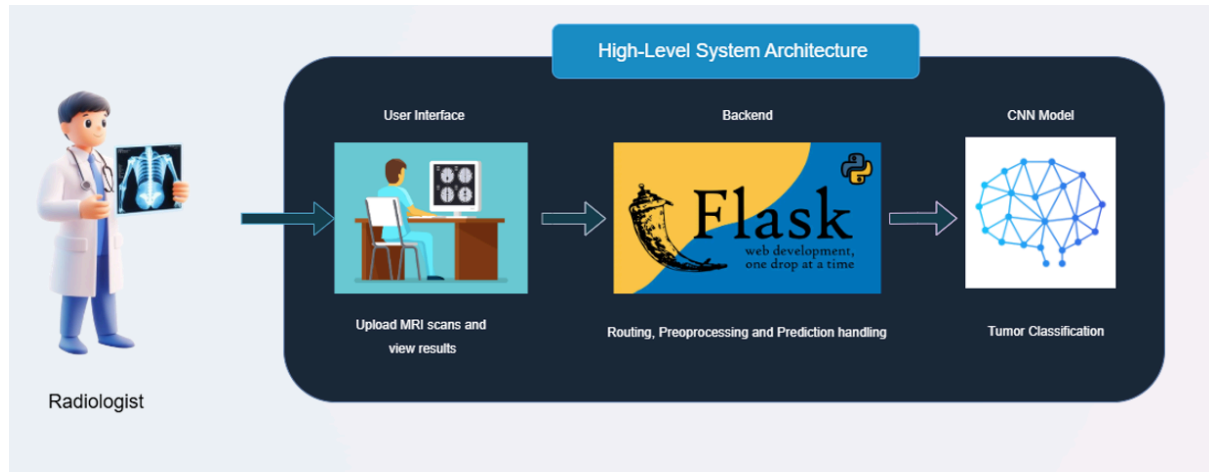


Figure 4.6 - High-level deep learning model architecture (Author's own work, created using Flowchart, images sourced from Freepik and Khandekar, 2023)

The proposed system architecture is a web-based application that uses HTML and CSS for the front-end interface, Flask for the backend to handle routing and process requests, and a transfer learning model based on deep learning model for tumour classification.

The system begins when the radiologist uploads an MRI scan through the webpage. The image is forwarded to the flask backend, where it's preprocessed before being passed to the deep learning model. The model then analyzes the MRI scan and generates a result, which is returned to the backend and finally displayed on the interface.

The architecture was chosen due to its simplicity and efficiency, as it handles smooth interaction between the interface, backend, and deep learning model. Additionally, it allows further improvements such as updating the model or adding additional features to the system.

4.7 Model Design

This section describes the design of the deep learning model used for brain tumor classification, including dataset preparation, processing techniques, and selection of model architectures. Convolutional Neural Networks (CNNs) are a type of deep learning model widely used for image classification because they can extract features directly from images (Minarno et al., 2022). They are particularly effective in medical imaging as they can identify patterns in MRI scans. The model was developed using transfer learning techniques to achieve high accuracy as possible while keeping the model efficient.

4.7.1 Dataset Preparation

The dataset for this project was obtained from Kaggle, published by Masoud Nickparvar. It consists of over 7,000 MRI brain tumor scans grouped into four classes: Glioma, Meningioma, Pituitary, and No tumor. The dataset was divided into training and validation sets using an 80/20 split. This allows the model to learn meaningful features from the training data while being evaluated on unseen data which lowers the risk of overfitting.

4.7.2 Preprocessing and Augmentation

The original dataset contained images of different dimensions. To ensure consistency with the model's architecture, all images were resized to a fixed dimension of 512×512 pixels. Preprocessing is a crucial step as it improves data quality and improves the model's predictions (Minarno et al., 2022). Data augmentation techniques such as horizontal flipping, rotation, and zooming were applied to increase data diversity.

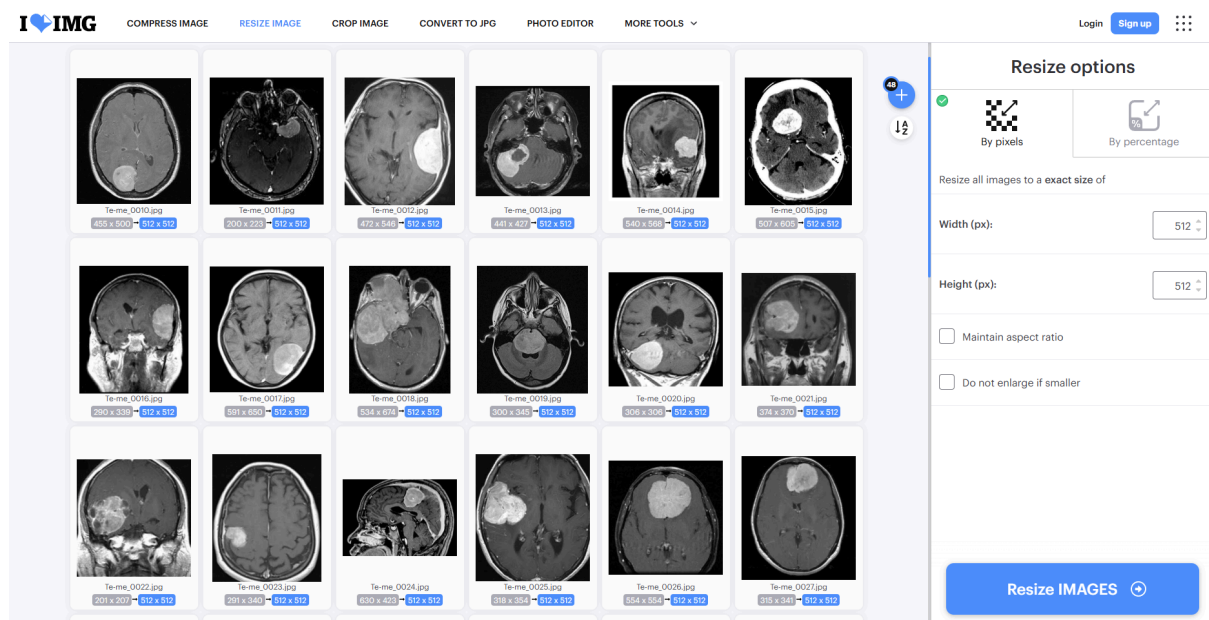


Figure 4.7.2 - Example of MRI images before and after preprocessing through resizing to a fixed dimension of 512×512 pixels

4.7.3 Model Architecture

Several pre-trained convolutional neural network architectures, including ResNet50, VGG16 and EfficientNetB0, were considered during the design phase due to their strong performance in handling image classification tasks (Anand et al., 2026; Poornima et al., 2025). The model is integrated into the system through a transfer learning approach, where the pre-trained network is adapted to classify MRI images into four categories.

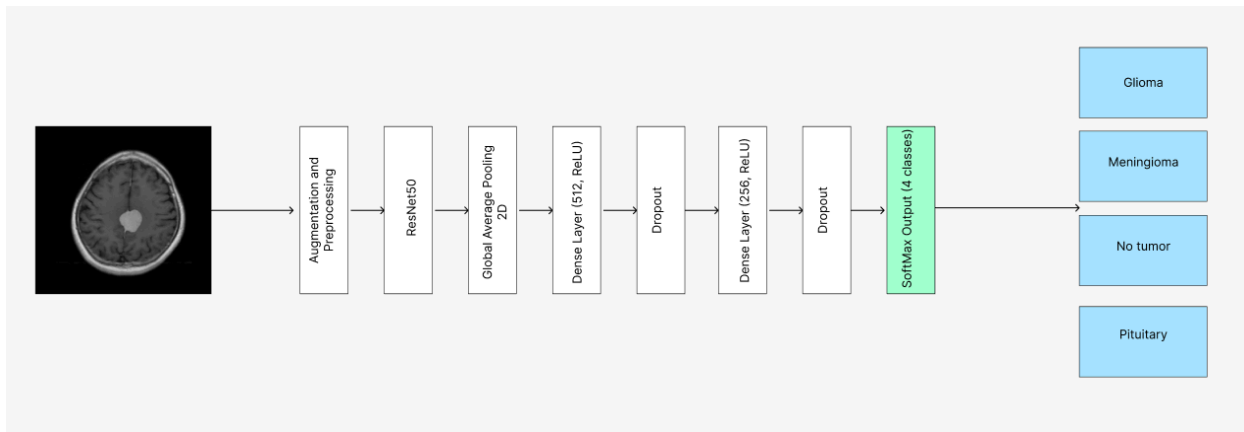


Figure 4.7.3 Proposed ResNet50 CNN model for brain tumor classification (Author's own work, created using Figma)

4.8 Web Application Design

This section describes the design of the web application, focusing on how users interact with the system and how requests are processed to generate tumour predictions.

4.8.1 Splash Screen

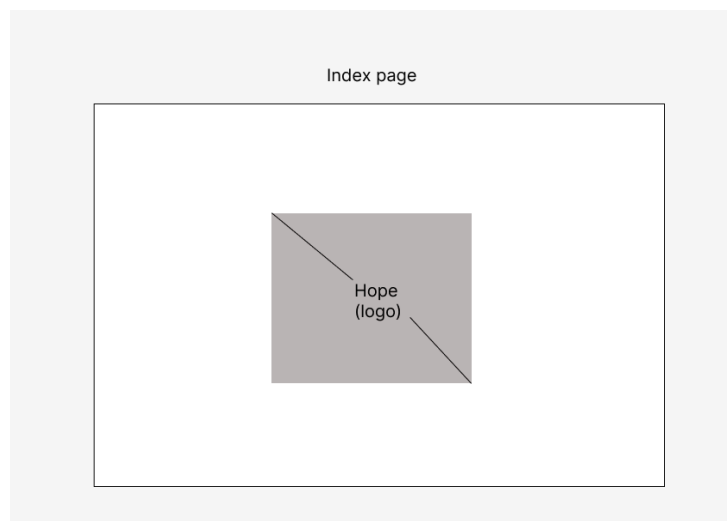


Figure 4.8.1 - Wireframe for Splash Screen

This screen is displayed when the application is launched.

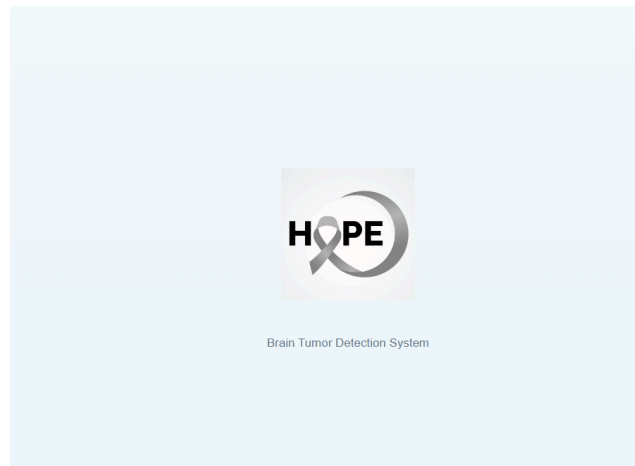


Figure 4.8.2 - Final Splash Screen Interface

The splash screen uses a grey ribbon logo, which represents brain cancer awareness. This was chosen to give the application a meaningful and relevant visual identity.

4.8.2 Language Selection Screen

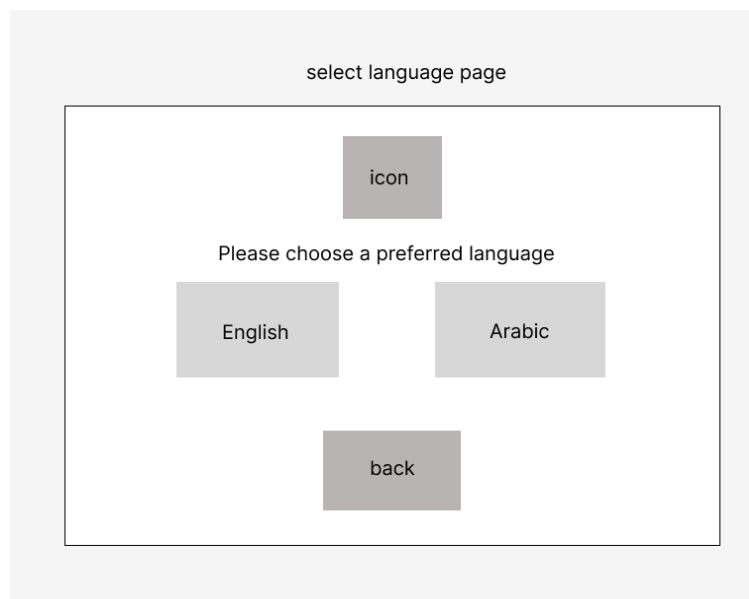


Figure 4.8.3 - Wireframe for Language Screen



Figure 4.8.4 - Final Language Screen Interface

This screen was designed to allow users to select their preferred language. Including both English and Arabic, which improves accessibility and ensures the system can be used by a wider range of users.

4.8.3 Main Menu

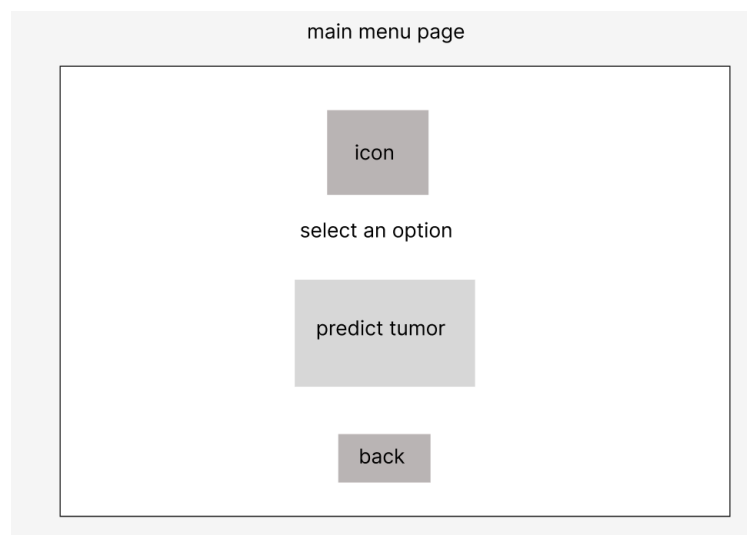


Figure 4.8.5 - Wireframe for Main Menu Screen

The main menu navigates the user to access the main functionality of the system, predicting a tumor.

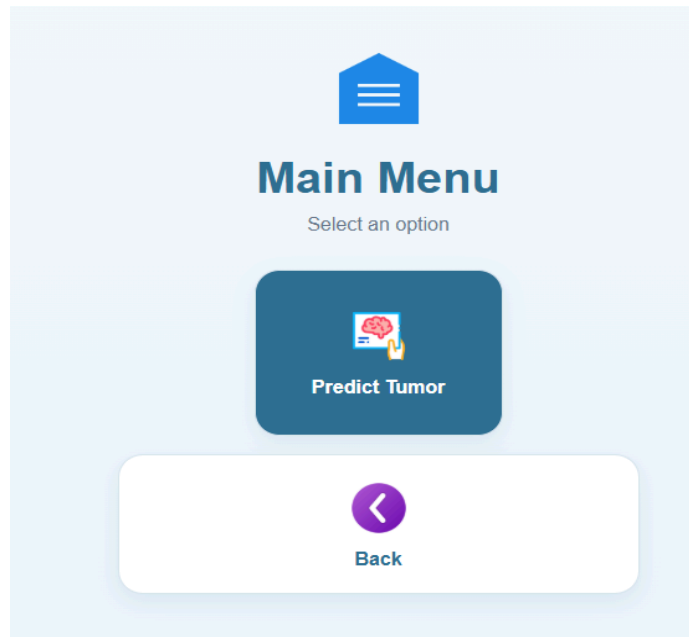


Figure 4.8.6 Final Main Menu Interface

The main menu was designed to be simple and focused on the core functionality of the system. The “Predict Tumor” option is clearly highlighted to guide users directly to the main task. The layout avoids unnecessary complexity, making it easier for users to navigate the system.

4.8.4 Prediction Screen

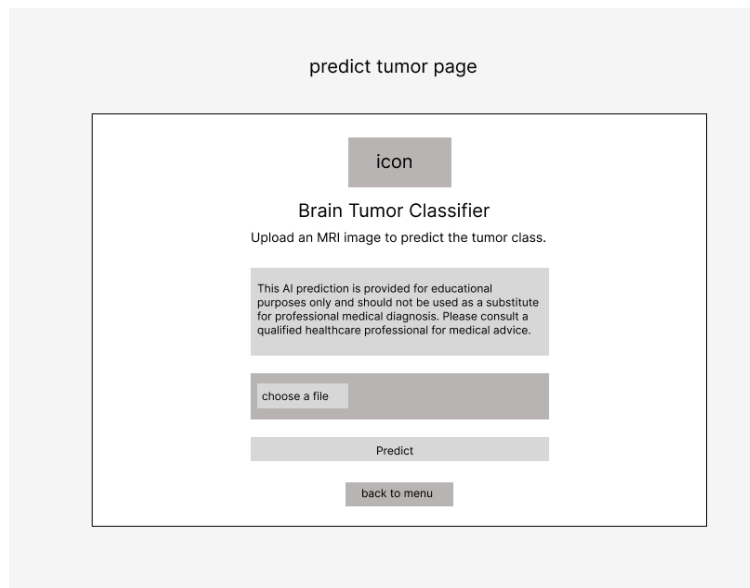


Figure 4.8.7 - Wireframe for Prediction Screen

This screen allows the user to upload an MRI screen and submit it for analysis.

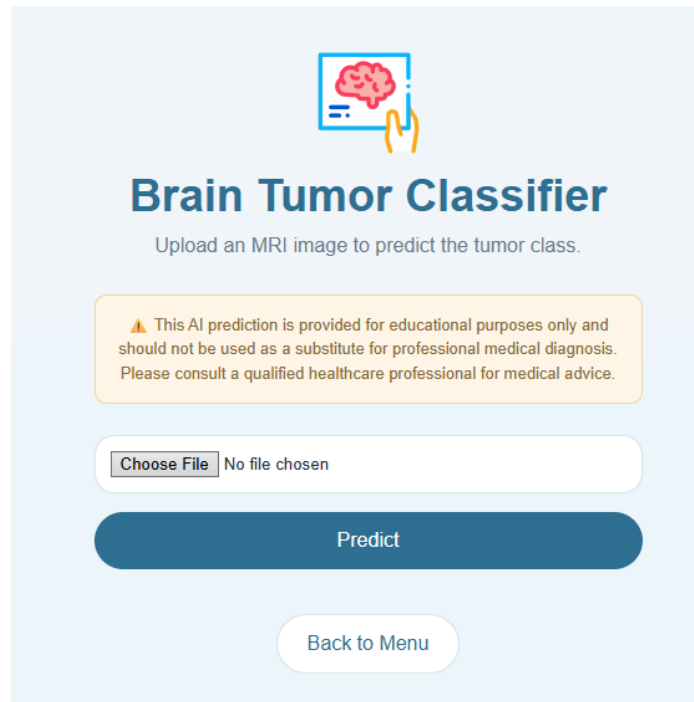


Figure 4.8.8 - Final Prediction Screen Interface

The prediction screen was designed to make the upload process straightforward. Users can easily select and submit an MRI image without facing any problems. The simple structure helps to reduce user effort and supports a smooth interaction with the system.

4.8.5 Result Screen

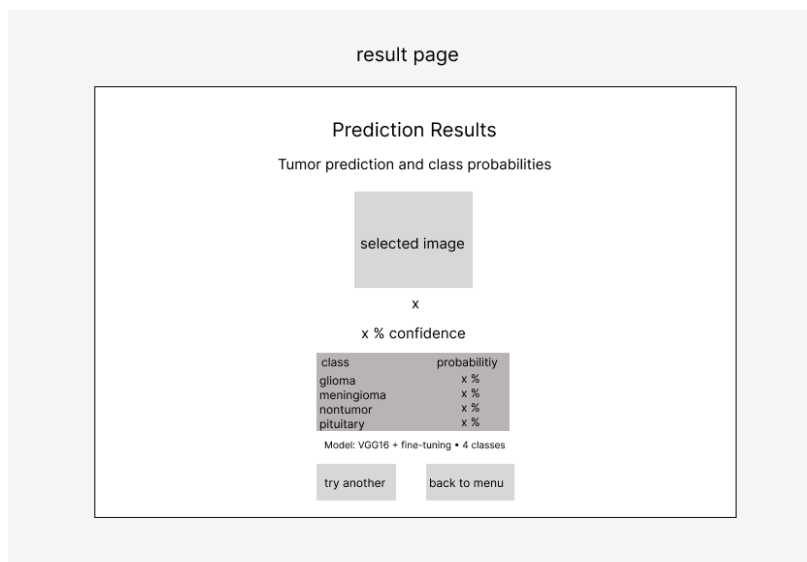


Figure 4.8.9 - Wireframe for Results Screen

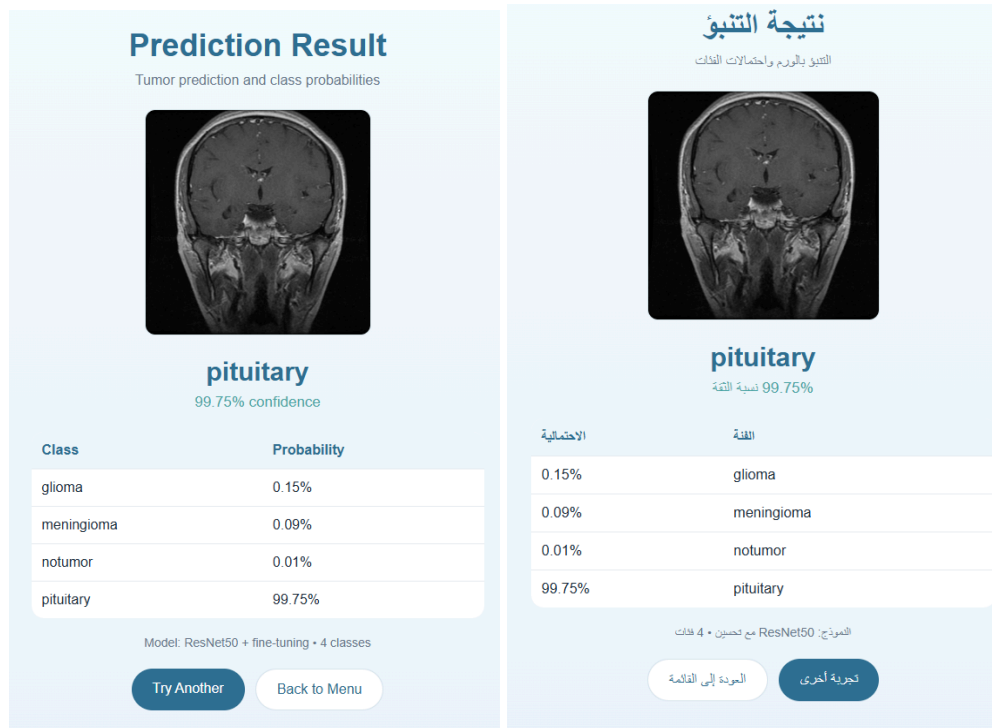


Figure 4.8.10 - Final Results Screen Interface with Arabic

This screen displays the prediction tumor type along with other possibilities. The user is able to retry a new prediction or return back to the main menu. Two interfaces are presented to show the similarity between English and Arabic.

4.9 Chapter Summary

This chapter presented the requirements, design and planning of the brain tumor detection system. It identified the radiologist as the primary user of the system and defined the functional requirements, such as uploading the MRI scan and receiving a prediction, and non functional requirements including performance, usability and reliability.

An iterative approach was proposed to support continuous improvement of the model and application. The model's architecture design and wireframes for the application were presented to demonstrate how the system is structured and used.

Chapter 5 - Development and Implementation

This section outlines the development and implementation stages of the brain tumour image classification model. Multiple models were tested prior to finalisation following an iterative methodology. The models were trained using Google Colab Pro with the runtime as A100 GPU, which was used for training due to its access to high-performance GPUs, allowing the model to run more quicker and efficiently.

The development and implementation of the model began with dataset preparation, followed by testing and fine tuning, implementing the final selected model and lastly integrated the final model into a Flask web application.

5.1 Model development and training

5.1.1 Dataset preparation

The Brain Tumour MRI dataset collected from Kaggle was used for this project, containing images divided into four classes: glioma, meningioma, pituitary and no tumour. Directories in the dataset are organised based on their relevant class labels. All the MRI images in the dataset were resized to 512x512 pixels before being inputted to the model. The dataset was split by a 80/20 ratio during training and validation to increase model efficiency. Additionally, data augmentation techniques, including horizontal flipping, rotations and zoom transformations, were applied to the model to prevent overfitting.

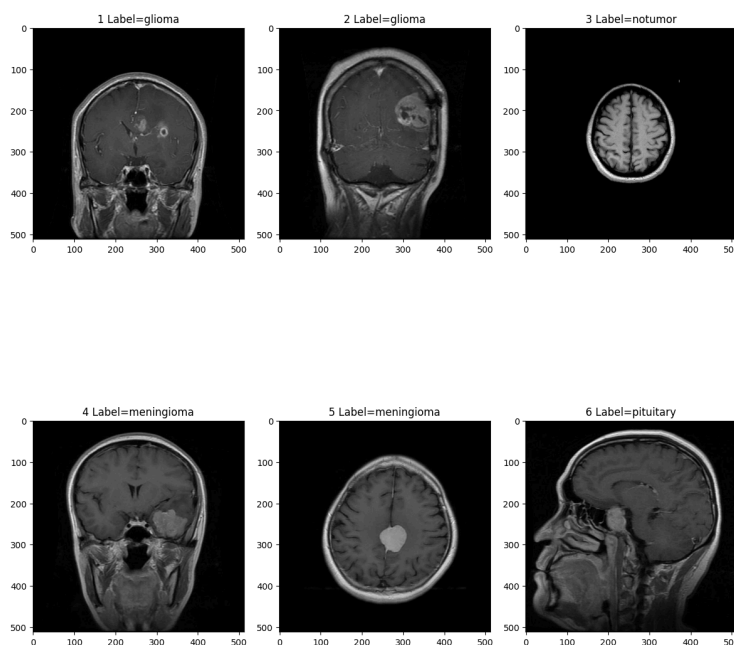


Figure 5.1.1 - MRI images from the dataset across four tumor classes (glioma, meningioma, nontumor and pituitary)

5.1.2 Model exploration

A transfer learning approach is utilised in this project and three separate pre-trained models were tested, which are ResNet50, EfficientNetB0 and VGG16. Their respective results were compared using validation accuracy and training efficiency as benchmarks.

ResNet50 is a strong contender as it uses residual learning, which helps it train deep networks addressing the vanishing gradient problem (Desanamukula et al., 2024). Evidently, ResNet50 also has achieved high accuracy in numerous published image classification tasks. However, it also has a larger number of parameters, making it computationally expensive.

Model: "sequential"		
Layer (type)	Output Shape	Param #
random_flip (RandomFlip)	(None, 512, 512, 3)	0
random_rotation (RandomRotation)	(None, 512, 512, 3)	0
random_zoom (RandomZoom)	(None, 512, 512, 3)	0
lambda (Lambda)	(None, 512, 512, 3)	0
resnet50 (Functional)	(None, 16, 16, 2048)	23,587,712
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0
dense (Dense)	(None, 512)	1,049,088
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 4)	1,028
Total params: 24,769,156 (94.49 MB)		
Trainable params: 1,181,444 (4.51 MB)		
Non-trainable params: 23,587,712 (89.98 MB)		

Figure 5.1.2 - Model summary of ResNet50 with parameters

The EfficientNet-B0 model is efficient and lightweight due to its fewer parameters and lower computational cost (Abdulkareem & Abdullah, 2026). However, this comparatively smaller model struggles to capture complex image features which is essential to classifying medical images.

Model: "sequential"

Layer (type)	Output Shape	Param #
random_flip (RandomFlip)	(None, 512, 512, 3)	0
random_rotation (RandomRotation)	(None, 512, 512, 3)	0
random_zoom (RandomZoom)	(None, 512, 512, 3)	0
efficientnetb0 (Functional)	(None, 16, 16, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense (Dense)	(None, 512)	655,872
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 4)	1,028

Total params: 4,837,799 (18.45 MB)
Trainable params: 788,228 (3.01 MB)
Non-trainable params: 4,049,571 (15.45 MB)

Figure 5.1.2. Model summary for EfficientNetB0 with parameters

VGG16 is also a reliable contender due to its simple and uniform model architecture which eases implementation for image classification tasks (Barman et al., 2024). However, similarly to the EfficientNetB0 model it is also computationally expensive due to a high number of parameters.

Model: "sequential"

Layer (type)	Output Shape	Param #
random_flip (RandomFlip)	(None, 512, 512, 3)	0
random_rotation (RandomRotation)	(None, 512, 512, 3)	0
random_zoom (RandomZoom)	(None, 512, 512, 3)	0
lambda (Lambda)	(None, 512, 512, 3)	0
vgg16 (Functional)	(None, 16, 16, 512)	14,714,688
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 512)	262,656
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 4)	1,028

Total params: 15,189,700 (57.64 MB)
Trainable params: 395,012 (1.51 MB)
Non-trainable params: 14,714,688 (56.13 MB)

Figure 5.1.2 - Model summary for VGG16 with parameters

Following the final testing results across these three models, ResNet50 was chosen as the final model, which is explored further in depth in Chapter 6: Testing and Evaluation.

5.1.3 Final model implementation

After the selection of the ResNet50 pre-trained model, it was loaded into the image classification model without its top layers, allowing it to extract features from the MRI image dataset.

Custom layers were then added on top of the base model. A Global Average Pooling layer was used to reduce the feature maps to a single average value to reduce Dense layers and overfitting (Munn & Pitman, 2022). This layer is followed by fully connected Dense layers with ReLU activation. Dropout layers were included to also further reduce the risk of overfitting. The final output layer consists of four output neurons with a SoftMax activation function.

Fine-tuning was then applied by unfreezing the last few layers of the ResNet50 model while keeping earlier layers frozen. This allowed the model to adapt to the MRI dataset while retaining general features learnt from ImageNet.

5.1.4 Training configuration

The model was compiled using the Adam optimizer with a learning rate of $1e-5$, which supports stable and gradual updates during training.

Categorical cross-entropy was used as the loss function, as the task involves multi-class classification. A batch size of 8 was selected to balance memory usage and training efficiency working with high-resolution MRI images.

To improve training control, callbacks were implemented. ModelCheckpoint was used to save the best-performing model based on validation loss, while EarlyStopping was used to stop training once performance improvements began to plateau (R et al., 2024).

The model was trained over multiple epochs, with validation accuracy and loss monitored throughout the process to observe fluctuations during the training process.

5.1.5 Model saving and export

The fine-tuned ResNet50 model after reaching its highest accuracy was then saved and integrated into a Flask-based web application, where the model is deployed to produce real-time predictions through a web interface.

5.2 Flask web application development

5.2.1 Project structure

The model was deployed in flask with a simple structure to keep everything organized. The core code was written in app.py, which handled the routes, image processing, and model predictions. The front-end, which consisted of HTML and CSS, was saved in template and static files, respectively.

The application runs using basic routes to navigate between the web pages such as main menu, language option, predication and results page. Additionally, when an MRI image is uploaded, it is preprocessed, converted to RGB format and transformed into an array. Lastly, the image is then passed into the trained model to generate a prediction.

5.2.2 Additional features

An additional feature added to the application was a language option, in which users can choose between English or Arabic. This ensures the application is not limited for English users only and improves accessibility for a wider range of users.

5.3 Web Application Interface

This section explains how the user interface, designed in Chapter 4, was implemented within the Flask application. The interface was developed using HTML and CSS, while Flask handled routing between pages such as the main menu, language selection, prediction, and results screens.

The application follows a simple workflow, where users upload MRI images through the prediction page. The image is then processed by the backend and passed to the trained model, and the results are displayed along with confidence scores. This ensures a smooth interaction between the user interface and the deep learning model.

5.4 Challenges and Solutions

During the time of developing the system, several challenges were faced both in the model and its integration into Flask. One of the initial issues occurred during the image processing stage. The dataset originally wasn't resized, which caused inconsistency during training. This was resolved by changing all the images in the dataset to a fixed size of 512 x 512 pixels, which improved the model's performance.

Another issue was related to how the model was trained. Initially, the model was split into two phases, Phase 1 would train the model and Phase 2 was applied with fine-tuning. However this approach caused an unnecessary re-running of the model which increased the computational costs and time. To solve this, the model was trained once with fine-tuning applied, which showcased much better results as the process was more efficient.

	precision	recall	f1-score	support
glioma	0.91	0.79	0.84	300
meningioma	0.80	0.74	0.77	306
notumor	0.93	0.96	0.95	405
pituitary	0.84	0.99	0.91	300
accuracy			0.88	1311
macro avg	0.87	0.87	0.87	1311
weighted avg	0.88	0.88	0.87	1311

Figure 5.4.1 Phase 1, 2 and without resizing the dataset equally (VGG16)

	precision	recall	f1-score	support
glioma	0.99	0.97	0.98	300
meningioma	0.98	0.96	0.97	306
notumor	0.99	1.00	0.99	405
pituitary	0.98	0.99	0.99	300
accuracy			0.98	1311
macro avg	0.98	0.98	0.98	1311
weighted avg	0.98	0.98	0.98	1311

Figure 5.4.2 Removal of phase 1 and 2 and resizing the dataset equally. (VGG16)

Finally, an issue was encountered for the Flask application. The main issue was related to loading the saved keras model files. The models saved from the notebook weren't loading or registering in Flask due to compatibility issues between Keras and TensorFlow versions. This was resolved by verifying the versions in Google Collab and Flask (PyCharm) were aligned, allowing the model to load and run properly.

5.5 Chapter summary

This chapter presented the development and implementation of the system, including model training, selecting the best model and integrating into Flask. Three pre-train models were explored before selecting ResNet50 as the final model. The model was integrated into a Flask web application, allowing users to upload MRI images and receive predictions through a simple interface, with additional features such as language selection. Several challenges were encountered during development, including preprocessing issues and deployment compatibility, which were resolved to ensure the system functions correctly. Overall, a working system was successfully developed and is ready for further evaluation in the next chapter.

Chapter 6 - Testing and Results

This section outlines the testing and evaluating the performance of the models, which includes a comparison of the results on unseen MRI images. The model's performance is also measured by metrics such as accuracy and loss. In addition, further analysis is demonstrated using a confusion matrix to understand the strengths and weaknesses of the system.

6.1 Experimental Setup

Following the dataset preparation described in Chapter 5, the model was trained and evaluated using images resized to 512×512 pixels. A batch size of 8 was selected as it provided a balance between memory usage and training performance. This is important, as appropriate batch size selection can significantly improve model performance (Ferhat Sarikaya, 2024).

All three models were trained using the same augmentation techniques, including randomflip(horizontal), randomrotation(0.05), randomzoom(0.05). In addition, fully connected Dense layers of sizes 512 and 256 with ReLU activation function, including L2 regularization (0.0001). Lastly, the final output layer used was a SoftMax activation function containing four neurons to classify the MRI images into one of the four tumor classes.

6.2 Training and Validation Results

6.2.1 VGG-16

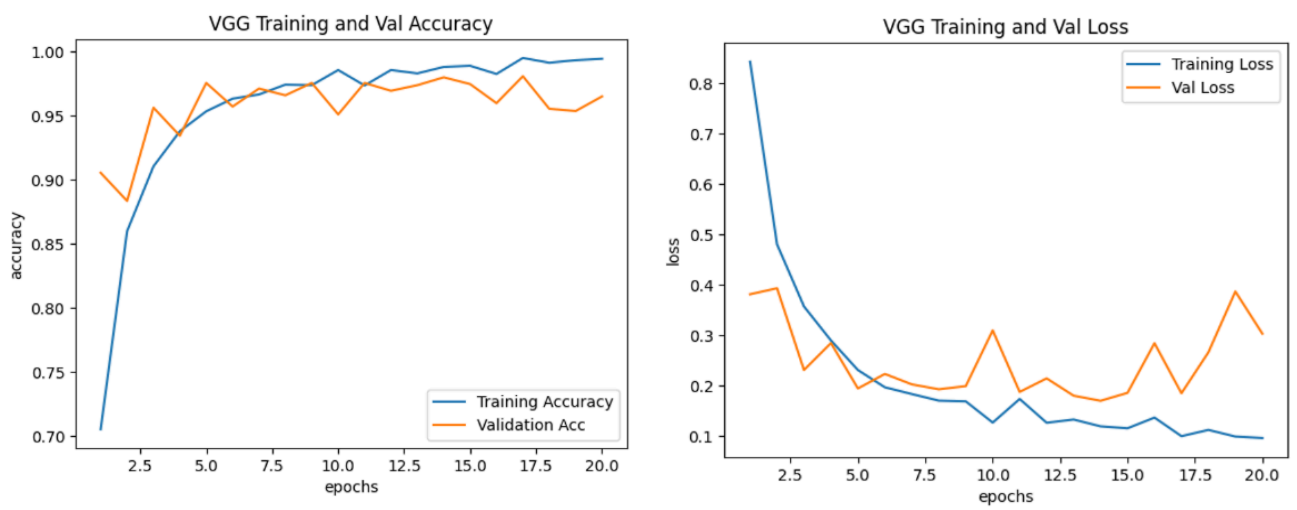


Figure 6.2.1 VGG16 training and validation (Accuracy and Loss)

The training and validation results for VGG16 shows that the model was able to learn very quickly at the start. Validation accuracy increased over 85.0% within the first epochs, which highlights that the model was able to learn and extract meaningful features from the dataset.

The training accuracy continued to improve reaching over 95.0%, which shows that the model trained the data very well. However, some fluctuations are seen in validation, which indicates that despite it learning well, the model isn't fully stable.

Overall, VGG16 demonstrated strong performance with high accuracy, but with slight uncertainty during training.

6.2.2 ResNet50

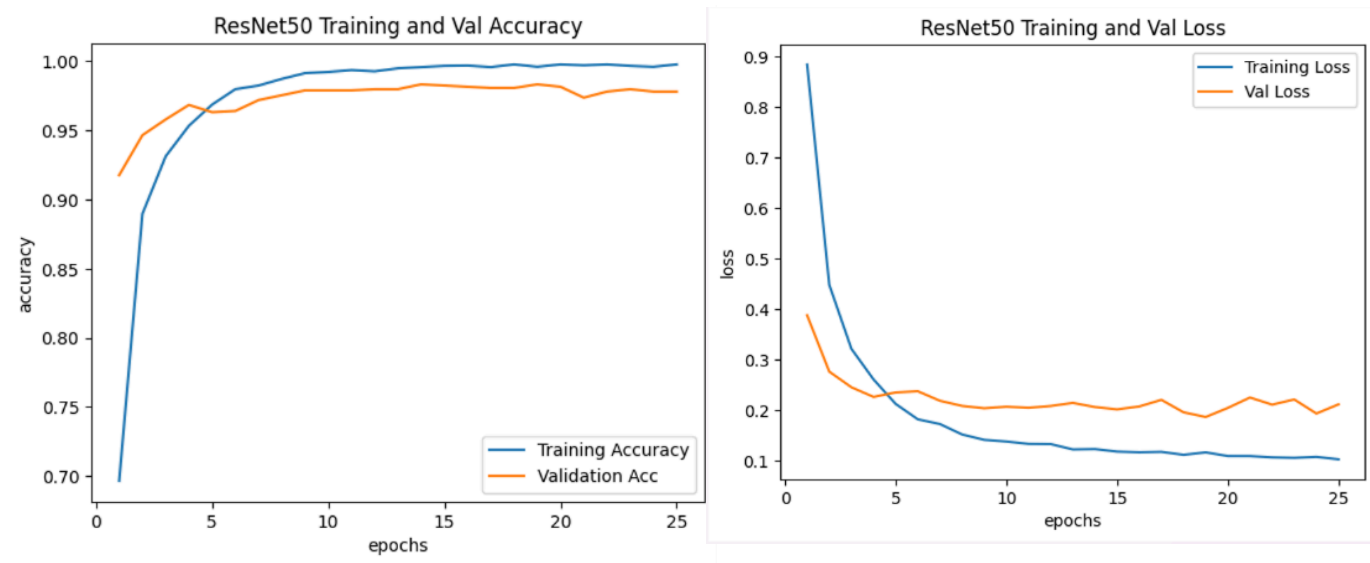


Figure 6.2.2 ResNet50 training and validation (Accuracy and Loss)

Similarly to VGG16, the training and validation accuracy shows that the model was able to learn quickly during the early epochs. During training the model continued to improve, even reaching close to 99.0%, while the validation remained around 97.0%. Which shows how effective the model was during training and validating on unseen data. However, the results for ResNet50 showed fewer fluctuations compared to VGG16, which indicates a more stable process.

In conclusion, ResNet50 demonstrates a more stable performance, which makes it the most suitable model for the system.

6.2.3 EfficientNetB0

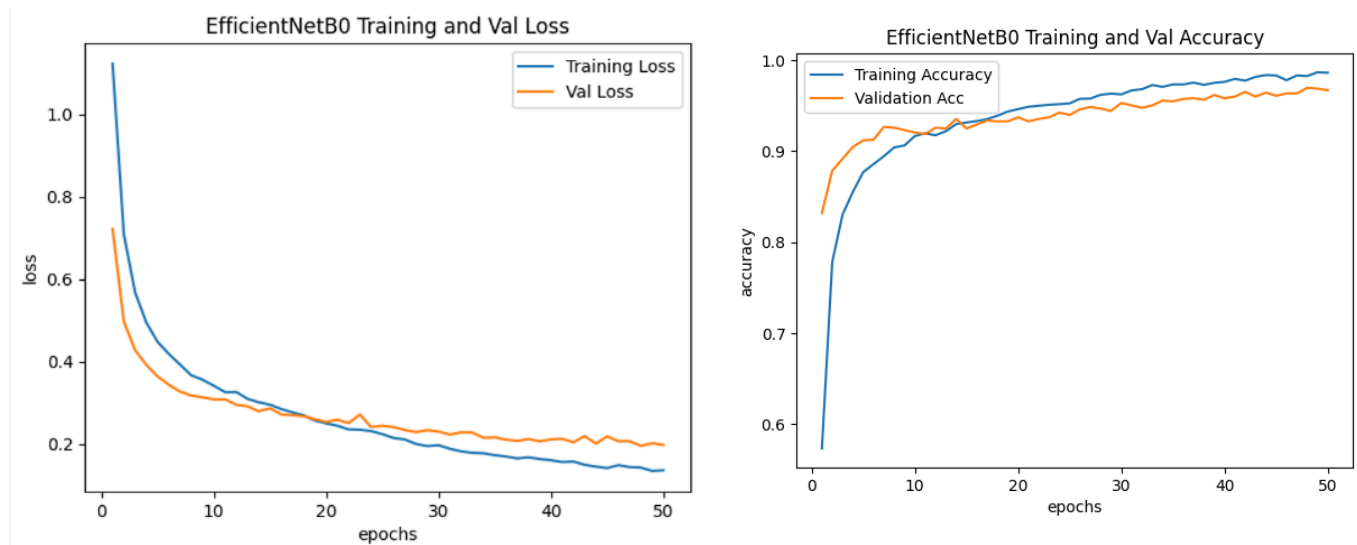


Figure 6.2.3 EfficientNetB0 training and validation (Accuracy and Loss)

The training and validation results show that the model was able to learn properly over time. Although, when comparing results with VGG16 and ResNet50, the learning process is much slower during the initial epochs. This highlights that the model needed more time to extract meaningful features from the dataset.

The training accuracy did increase, reaching values close to 98.0%, while the validation accuracy improved around 95%. This shows that the model was also able to learn well to unseen data.

Overall, EfficientNetB0 was able to perform well, but its accuracy was slightly lower compared to ResNet50 and VGG16, making it not suitable for the system.

6.3 Testing Results

ResNet50 achieved the highest accuracy of 99.0%, which is followed by VGG16 with 98.1% and lastly with EfficientNetB0 which achieved an accuracy of 96.5%. These results show how well the models performed with high accuracy across the test dataset. Based on these results, ResNet50 was chosen as the final model, as it demonstrated the highest test accuracy, which would be the best for the overall system.

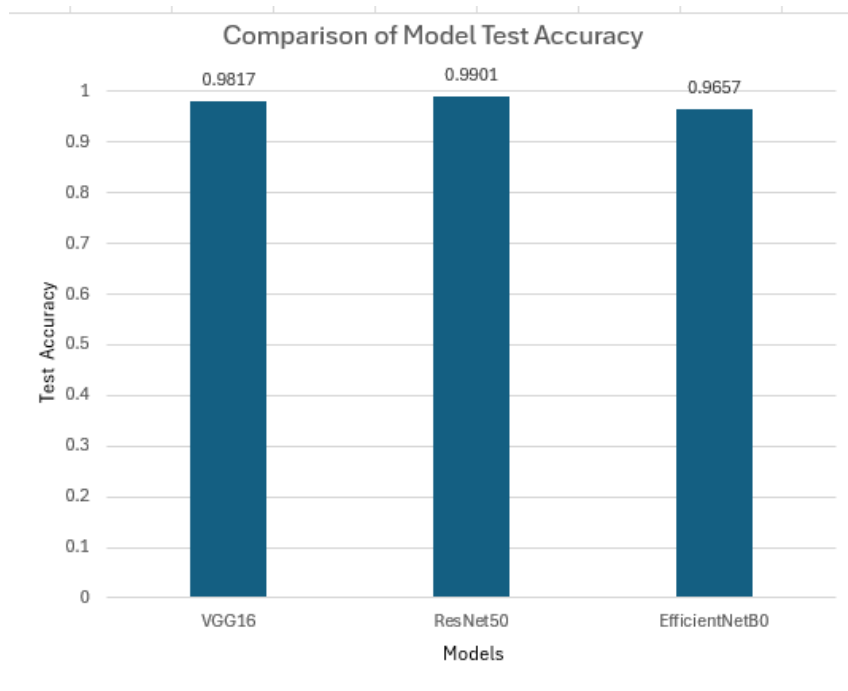


Figure 6.3 Comparison of testing results between the models

6.4 System and Input Testing

6.4.1 Invalid Input Testing

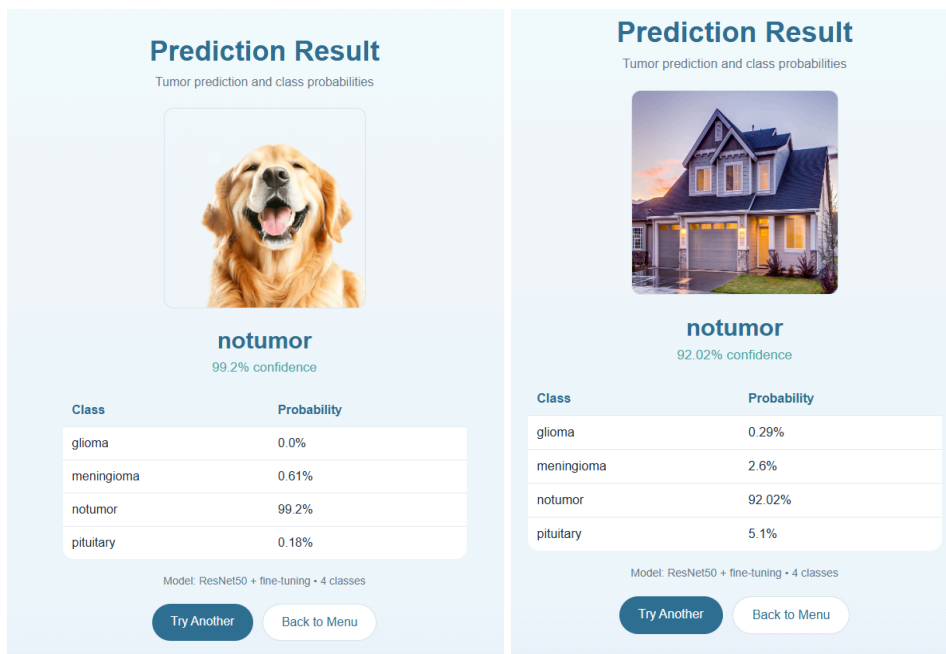


Figure 6.4.1 - Invalid input testing for images of a dog and house

Invalid input testing was conducted to evaluate how the system behaves when non-MRI images are uploaded. As shown in Figure 6.4.1, the system will generate predictions for these inputs, often classifying them as “nontumor” with high confidence, although in some cases a tumor class may be assigned. This indicates that the model is not fully able to distinguish between MRI scans and unrelated images. Therefore, this will be an additional feature, such as an “empty class” to be added in future work to improve the systems reliability.

6.5 Front-End Validation

HTML and CSS validation was carried out using tools such as W3C Markup Validation Service and CSS Validation Service to ensure that the web interface is following a regular standard. As shown in figure 6.5.1 and 6.5.2, the results indicate no major errors or issues. Which confirms that the web interface is valid and consistent across the application.

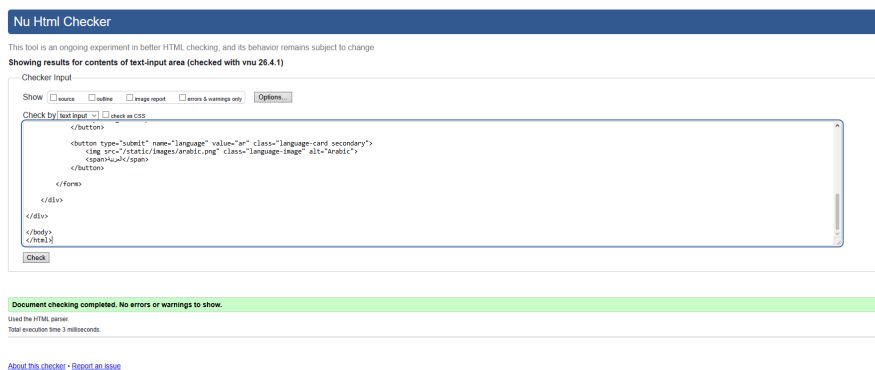


Figure 6.5.1 HTML validation for language.html



Figure 6.5.1 CSS validation for language.html

6.6 Confusion Matrix and Classification Analysis

ResNet50

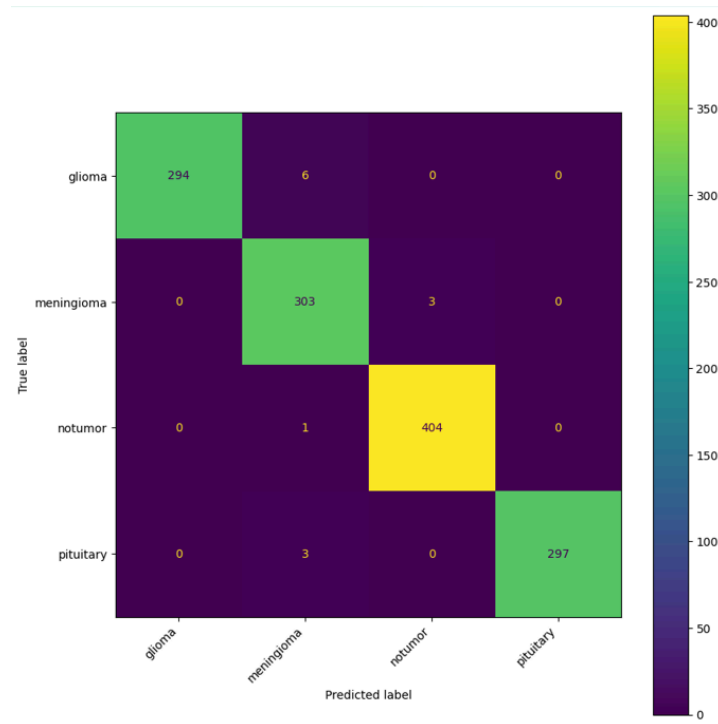


Figure 6.6 - Confusion Matrix for ResNet50

A confusion matrix was used to measure the model's performance across the tumor classes, helping to identify which classes the model could classify correctly and others weakly. Based on the results, the model performs the best in "no tumor" cases, with one misclassification, where it identifies as meningioma. However, minor confusion is presented for meningioma and glioma, which may have been sharing similar features in MRI images.

Therefore, this demonstrates that the model achieved high accuracy, despite having a small difficulty in distinguishing between similar tumor types.

6.7 Chapter Summary

This chapter presented the testing and evaluation models used in the system. The results showed that ResNet50 was able to perform remarkably well across training, validation and testing. Additionally, a confusion matrix was used to further analyze the model's classification performance, which highlighted a strong accuracy although having minor misclassifications.

Chapter 7 - Critical Evaluation

This section evaluates the requirements that were considered before starting the project, which includes the objectives, strengths and weaknesses. Including the challenges encountered during the development of the system and any features that weren't fully implemented due to time constraints. Lastly, ethical and practical considerations are addressed, along with key lessons learned throughout the project.

7.1 Achievement of objectives

Objective 1: Conduct a literature review on brain tumor detection and find suitable deep learning approaches

This objective was successfully completed during the early stages of the development. I reviewed multiple research papers on convolutional neural networks (CNN), transfer learning and MRI Brain Tumor classification. This provided me with a better understanding of key techniques and limitations within the research. Lastly, the design and development decisions wouldn't have been possible without the literature review.

Objective 2: Identify and prepare a suitable dataset

This was one of the most important tasks, to which I also achieved. I chose a publicly available dataset from Kaggle. Which contained over 7,000 MRI scans across 4 categories, being glioma, meningioma, pituitary and no tumor. Additionally, before preparing the model, I had to resize all the images in the dataset to a fixed size of 512 x 512 pixels.

Objective 3: Find and compare different deep learning models

I was able to achieve this, to which I found three suitable deep learning models, which were ResNet50, VGG16 and EfficientNetB0. I had to go through many different iterations when designing the models. As each model was evaluated based on training, validation and testing. This comparison provided me a clear understanding of which model would be the most suitable for the system in the end.

Objective 4: Develop a deep learning model and achieve an accuracy above 95%

After comparing the three models, I chose ResNet50 as my final model. Which I'm proud to say I achieved a testing accuracy of 99.0%, which is above my targeted goal. I implemented various techniques such as augmentation, regularization and fine-tuning to make it possible.

Objective 5: Develop a user interface for the system

Although this objective was aiming at a different interface system such as Android Studio, I was still able to successfully achieve this goal. I went for a different approach by implementing it in Flask. To which I also previously have done the basic sketches on the user interface and was able to complete the foundation of the system.

Objective 6: Additional features for the system

I was able to implement one of the two features, which was the language selection; the implementation for the consult form was later discarded due to a change of primary actors for the system being radiologist. Thus, I was still able to achieve this goal as well.

Objective 7: Testing of the final system

This objective was successfully achieved, the flask application runs locally without any major bugs or errors to be discovered. Lastly, the model was tested with a different dataset which was selected from Mendeley Data to evaluate the performance on unseen data. The results obtained from this confirms that the system performs well.

7.2 Strengths of the project

One of the main strengths of the project is the high accuracy achieved by ResNet50, which was achieved at 99.0%. This exceeded the initial planned target and demonstrated that the model is able to classify MRI images effectively. Additionally, the integration of the model into Flask allowed the system to function as a reliable solution that could be practical for use by radiologists.

Another strength of the project is the use of transfer learning, which enabled the model to achieve high performance despite limited computational resources and dataset size. This approach reduced training time while still maintaining strong predictive capability. Furthermore, the system provides confidence scores alongside predictions, which improves transparency and allows users to better understand the model's decision-making process.

7.3 Limitations and weaknesses

Due to time constraints, there were limitations that I wasn't able to solve and will need be further considered,

- The dataset being used was obtained from one source, which may affect how the model predicts with real-data.
- Minor misclassifications are observed in the system between glioma and meningioma.

7.4 Deviations from Project Plan

During the development of the project, one key deviation occurred from the original plan, where the user interface (UI) was supposed to be in Android Studio with TensorFlow Lite. However, this approach was too time-constraining, as the IDE was often unresponsive and my laptop wasn't sufficient to run it smoothly.

Another alternative was using FastAPI, but I was unable to find any sufficient resources to properly learn how to implement a classification model. As a result, I decided to use Flask, as it provided more resources for integrating the deep learning model into a web application.

7.5 Ethical, legal, and practical concerns

The proposed system is designed for educational use only and should not be considered as a final evaluation, as mentioned in the flask application as well. Moreover, the system is not replacing radiologists, as identifying brain tumors goes through a very careful process, where any risks can't be taken.

From an ethical point of view, it is important for users to understand the limitations of the system. For example the dataset being used was publicly available, where in real case scenarios the data needs to have strict privacy and protection, to ensure there won't be any data leaks.

7.6 Lessons learned

I was able to learn a lot about deep learning and how models are used in practice. One of the main things was learning how important the preprocessing and fine-tuning stages are, as they had a huge impact on my system.

I also learned how most of my objectives did not meet the expected deadline, as time constraints and issues faced took me longer than expected. This helped me to understand the importance of working iteratively and testing different solutions before finalizing my final results.

Another important lesson was time management, as some parts of the project took me longer than expected, especially when dealing with bugs and integration issues, which showed me the importance of planning and prioritizing my tasks properly.

Overall, this project helped me gain a better understanding of both model development and system integration, which I hope will give me the confidence that I need to work on similar projects in the future.

7.8 Reflection on the first two dissertation deliverables

The initial dissertation deliverable, including project proposal and literature review went really well. It helped to define the overall direction of my project. However, early decisions, such as developing the system with AndroidStudio, was later discarded due to personal limitations. Overall, these deliverables helped to understand the need to adapt the project based on implementation challenges.

7.9 Chapter summary

This chapter discussed the objectives that I've achieved on starting the project. Most of my objectives were achieved, including achieving a high accuracy and integrating into Flask. The strengths and weaknesses of the system were discussed, along with the main challenges that were encountered during development and how they were resolved. Discussing the features that weren't fully implemented due to time constraints. Additionally, a comparison with existing research was also carried out to evaluate the performance of my model against other approaches.

Lastly, ethical concerns of the system were considered and the lessons that I learned throughout the process. This chapter highlights both the success and areas where improvements could have been made in the system.

Chapter 8 - Conclusions

8.1 Overall summary

This project was aimed to deliver a successful brain tumor classification system using deep learning techniques applied to MRI images. Multiple convolutional neural networks (CNN) were experimented, including VGG16, ResNet50, and EfficientNetB0, while using a transfer learning approach.

The final model selected for the system was ResNet50, as it achieved an accuracy of 99%, demonstrating a strong performance in classifying brain tumor images into 4 categories. Additionally, the model was successfully integrated into a Flask application, allowing users to upload images and receive predictions. Overall, the project was able to achieve its main objective, achieving a high accuracy and developing in an application.

8.2 Key Findings

The results from this project show that ResNet50 was the most effective model to classify brain tumor images, although all three models performed well. Compared to VGG16 and EfficientNetB0, ResNet50 was more stable during training and better at generalisation, which made it suitable for the system. Another important key finding is data consistency, as initially irregular image sizes negatively impacted the model to train poorly. However, when resizing occurred by converting the dataset to 512 x 512 pixels and applying techniques such as augmentation, it improved the models accuracy.

Additionally, using transfer learning and fine-tuning was sufficient enough to achieve high results, this demonstrates that complex techniques are not always required. However, the confusion matrix indicates that despite achieving a 99% accuracy, the 1% causes minor misclassifications between tumors like glioma and meningioma. This might also suggest similar features are presented in these cases, making it harder to predict. Lastly, the model was trained using a single dataset, which may not perform well, limiting it to predict real medical data.

8.3 Future work

I plan on improving my system in the future, such as adding an empty class, where non brain tumor MRI images are detected and further key improvement would be to use multiple datasets, which would increase data diversity and models ability to predict different MRI images. Another improvement is highlighting the tumor by using segmentation techniques, which would improve the use of the flask application. In addition, the current system runs locally, so I could focus on deploying the application online, making it accessible to many users. Lastly, the flask application is only a prediction system. I plan on adding a login page with a secure database, where radiologists are able to forward MRI scans that they are not able to diagnose properly.

8.4 Comparison with State-of-Art results

After completing the model, I further evaluated my results with other research papers that have used the same dataset. A study by Rahman et al. (2025) applied several techniques to the MRI images before classification such as Local Binary Patterns (LBP), Gabor filters and Discrete Wavelet Transform. Furthermore, these features were used with different models including CNN and Support Vector Classifier (SVM), which achieved an accuracy of 98.9%.

Another study tested by Uçar and Kurt (2026) demonstrated two approaches, a custom CNN called OkanNet and a transfer learning model using ResNet50. In their final evaluation, they used preprocessing techniques such as resizing the images to 224 x 224 pixels, conversion from grayscale to RGB and data augmentation were all applied before training. Overall, the ResNet50 achieved an accuracy of 96.49%, while OkanNet achieved 88.10%.

Further studies by Tang et al. (2025) proposed a deep learning model which combines the spatial blocks from VGG and residual blocks from ResNet. The overall goal was to be able to capture both local and global features from MRI images without needing the use of tumor masks. In the end, their model achieved an accuracy of 97.29%.

Lastly, a study by Puspita et al. (2025) proposed a deep learning model which was combined with a soft voting ensemble learning strategy to classify the MRI images. In their strategy, multiple deep learning models such as ResNet50, VGG16, MobileNet and DenseNet21 were combined to improve the overall performance. During the preprocessing stage, their images were also resized to 512 x 512 pixels similar to my approach. At the end, they were able to achieve an accuracy of 97.67%.

In conclusion, I was able to achieve an accuracy of 99.0% using a transfer learning approach for ResNet50. This highlights that a fine-tuned model can significantly perform better without applying several approaches while maintaining a simple system.

Author	Models	Accuracy
Rahman et al. (2025)	CNN and SVM	98.9%
Uçar and Kurt (2026)	ResNet50(transfer learning) and OkaNet(custom CNN)	96.49%(ResNet50) 88.10%(OkaNet)
Tang et al. (2025)	VGG blocks combined with ResNet blocks	97.29%
Puspita et al. (2025)	ResNet50, VGG16, MobileNet, DenseNet121 (soft voting)	97.67%
Current proposed system	ResNet50 (fine-tuning)	99.0%

Table 8.4 - Comparison of research results with the same dataset

8.5 Professional and Ethical Considerations

The project highlights the responsibility and importance of using such systems, as it is purely designed for educational purposes only and should not be considered as a replacement for medical diagnosis by traditional means. As careful considerations are applied with machine learning in sensitive domains such as healthcare.

Reference list

Abdusalomov, A. B., Mukhiddinov, M., & Whangbo, T. K. (2023, 18 August). Brain Tumor Detection Based on Deep Learning Approaches and Magnetic Resonance Imaging. *Cancers*, 15(16), 4172. <https://doi.org/10.3390/cancers15164172>

Abdulkareem, M. H., & Abdullah, H. A. (2025, 15 December). Efficientnet-B0-based image classification: architecture, training, evaluation and visualization. *IET Conference Proceedings*, 2025(41), 37–44. <https://doi.org/10.1049/icp.2025.4377>

Anand, V., Khajuria, A., Pachauri, R.K. and Gupta, V., (2026, 27 January). *Multi-class classification of brain tumors using optimized CNN and transfer learning techniques*. *Scientific Reports*, 16, p.4709. <https://doi.org/10.1038/s41598-025-34806-6>

Barman, D., Satheesh, A., Vanlalpeka, J., Bora, A., & Nandi, G. (2024, 31 July). Enhancing Brain Tumor Detection Through Deep Learning: A Comparative Study of CNN and Pre-trained VGG-16 Models. *Lecture Notes in Networks and Systems*, 87–98. https://doi.org/10.1007/978-981-97-3601-0_7

Chattopadhyay, A., & Maitra, M. (2022, 4 March). MRI-based brain tumour image detection using CNN-based deep learning method. *Artificial Intelligence in Brain Informatics*. <https://doi.org/10.1016/j.neuri.2022.100060>

Desanamukula, V. S., Dharma Teja, T., & Rajitha, P. (2024, 7 June). *An in-depth exploration of ResNet-50 and transfer learning in plant disease diagnosis*. In Proceedings of the 2024 International Conference on Inventive Computation Technologies (ICICT) (pp. 614–621). IEEE. <https://doi.org/10.1109/ICICT60155.2024.10544802>

EIT Health. (2024, 18 September). Machine learning in healthcare: Uses, benefits and pioneers in the field. <https://eithealth.eu/news-article/machine-learning-in-healthcare-uses-benefits-and-pioneers-in-the-field/>

Freepik. (n.d.). *Images retrieved from Freepik*. [Image] <https://www.freepik.com>

GeeksforGeeks. (2025, 30 September). Waterfall model – software engineering. <https://www.geeksforgeeks.org/software-engineering/waterfall-model/>

Khandekar, A. (2023). *Flask for beginners*. [Image] Retrieved from <https://aseemkhandekar.hashnode.dev/flask-for-beginners>

K.V, C., & King, G. R. G. (2023). Brain tumour classification: a comprehensive systematic review on various constraints. *Computer Methods in Biomechanics and Biomedical Engineering: Imaging & Visualization*, 11(3), 517–529. <https://doi.org/10.1080/21681163.2022.2083019>

Minarno, A. E., Sasongko, B. Y., Munarko, Y., Nugroho, H. A., & Ibrahim, Z. (2022, 22 September). *Convolutional neural network featuring VGG-16 model for glioma classification*. <http://dx.doi.org/10.30630/joiv.6.3.1230>

Nickparvar, M. (2021). Brain Tumor MRI Dataset [Data set]. Kaggle. <https://doi.org/10.34740/KAGGLE/DSV/2645886>

Munn, M., & Pitman, D. (2022). *Explainable AI for Practitioners*. O'Reilly Media, Inc.

Poornima, P.U., Tamilalagan, S., Praveen Raj, D., Pongiannan, R.K., Jishnu, K.S. and Thanalakshmi, S., (2025, 26 July). *Brain tumor classification using ResNet-50 and EfficientNet-B0*. Springer.

Puspita, K., Ernawan, F., Alkhalifi, Y., Kasim, S., & Erianda, A. (2025, 30 September). *Brain tumor classification based on convolutional neural networks with an ensemble learning approach through soft voting*. <http://dx.doi.org/10.62527/joiv.9.5.4609>

Rahman, A., Hayat, M., Iqbal, N., Alarfaj, F. K., Alkhalaf, S., & Alturise, F. (2025, 11 August). *Enhanced MRI brain tumor detection using deep learning in conjunction with explainable AI SHAP based diverse and multi feature analysis*. <https://doi.org/10.1038/s41598-025-14901-4>

Rath, A., Mishra, B. S. P., & Bagal, D. K. (2024, 21 December). ResNet50-based deep learning model for accurate brain tumor detection in MRI scans. <https://doi.org/10.1016/j.nexres.2024.100104>

R, M. T., Thakur, A., Gupta, M., Deepak Kumar Sinha, Kritika Kumari Mishra, Vinoth Kumar Venkatesan, & Suresh Guluwadi. (2024, 22 January). *Transformative Breast Cancer Diagnosis using CNNs with Optimized ReduceLROnPlateau and Early Stopping Enhancements*. <https://doi.org/10.1007/s44196-023-00397-1>

Sarikaya, F. (2024, 11 November). *Batch size selection in deep learning: A comprehensive analysis of training dynamics and performance optimization*. <https://zenodo.org/records/14082377>

Tang, M., Zhou, X., & Li, Z. (2025, 29 November). *Brain Tumour Classification Model Based on Spatial Block–Residual Block Collaborative Architecture with Strip Pooling Feature Fusion*. <https://www.mdpi.com/2313-433X/11/12/427>

tutorialspoint. (n.d.). SDLC - Iterative Model. Retrieved December 20th, 2025, from https://www.tutorialspoint.com/sdlc/sdlc_iterative_model.htm

Uçar, O., & Kurt, M. (2026, April 1). *OkanNet: A lightweight deep learning architecture for classification of brain tumor from MRI images*. <https://arxiv.org/html/2604.01264v1>

Zamil, M. Z. H., & Lee, B. J. (2025, 2 June). Enhancing brain tumor detection using a CNN-powered Android application. <https://doi.org/10.1109/ISDFS65363.2025.11012085>

Appendix A: Project Proposal

1. Introduction

1.1 Aim - To develop an app for an image detection system for identifying four types of brain tumours in MRI scans. (*Glioma, meningioma, pituitary and no tumor*).

1.2 Background Study

Brain tumors are a serious neurological condition in which the growth of cells in the brain proliferates in an uncontrollable way. (NHS, 2023). There are more than 120 different types of brain tumors, which are distinguished by where they appear and what kind of cells they are made up of. (Johns Hopkins Medicine, n.d.). Tumors are either benign (non-cancerous) or malignant (cancerous), and it needs to be detected early if it's either one of them. (Mayo Clinic, 2024).

The timing on diagnosing a brain tumour is very crucial, as catching it late, can lead to having devastating outcomes. It can cause serious problems within the skull since the tumour will expand the brain, and the skull wouldn't be able to provide enough room for it. (Johns Hopkins Medicine, n.d.). If the tumour grows near to parts of the brain that are essential for movement, speech, behaviour and vision, it could severely impair these functions. (UNC Health Rex, n.d.).

Traditionally, the burden of identifying a tumour in medical images depended on radiologists, and the manual interpretation is time-consuming and prone to human error. Magnetic Resonance Imaging (MRI) has become a significant tool in providing detailed insights into the anatomy and pathology of the brain. (Mahesh et al., 2024). A crucial task when using MRI scans in such medical applications is the detection and segmentation of brain tumors, which require extensive data and analysis. Additionally, it presents various challenges such as image noise, low contrast, low borders, shifting intensities inside tissues, and tissue-type variation. (Kaifi, 2023).

According to World Health Organization, deaths caused by brain and nervous system cancers in the USA in 2022 are more common in ages of 55-74 between man and women,

the highest being in man. (WHO, n.d.), and over 300,000 new cases of brain and central nervous system (CNS) tumours are diagnosed annually worldwide. (aixial, n.d.)

1.3 Beneficiaries

This project is aimed towards radiologists and patients, which the radiologists being the client, responsible for analysis MRI scans, and the customers being the patients, who seek treatment for their brain tumor and wishing for a diagnosis.

According to Najjar (2023), “The advancements brought about by these AI-centric methods have amplified the precision and speed of diagnosis, thereby amplifying the competency of radiologists and raising the bar of patient care”. Developing this application will also allow patients who wish to receive educational feedback of their diagnosis, which would allow them to better understand their conditions and treatment. This will benefit them from having clear and strong communication between patients and doctors.

1.4 Personal Motivation

This project holds both personal and academic significance to me. On a technical level, I’ll be gaining hands on experience using Python for deep learning, in addition to training with CNNs, processing medical images and developing a mobile application. These experiences will improve my problem-solving and programming skills and introduce new skills in Artificial Intelligence (AI). This will bridge a gap between my academic and industry practices, providing practical experience which will benefit me to apply it with AI applications.

On a personal level, I’m motivated to research on topic as it stems from my late mother’s battle with brain cancer. My understanding of brain tumors at the time was limited, and undertaking this project would allow me not only to expand my knowledge but also to honor her by finding solutions and advancements in medical imaging of brain tumors using Artificial Intelligence (AI) for other patients.

2. Objectives

- I will read and summarize 2-3 research papers weekly throughout November 2025 to note down key gaps and differences between each report that aligns with the aim of my project.
- After I've completed reading research papers, I'll try to find a suitable dataset for my project before I start writing my literature review.
- I will complete a simple image classification model between October 24-31, 2025, to build a strong foundation before starting on the final project.
- Towards the last week of November, from November 23rd, 2025. I'll start writing on my literature review, which I will complete before December 11th, 2025.
- Once I've completed the literature review, I will begin writing, implementing and testing the main deep learning model and functions of the mobile application. I will need to complete the rest of the dissertation by the end of March 2026.
- I plan to develop and train a deep learning model that can classify MRI images of brain tumors into four categories, with a minimum test accuracy of 95% by the of March 2026.
- I will develop a mobile application as the user interface (UI), this will allow users to upload MRI images and receive automated feedback by the end of March 2026.
- I will also include features for the app such as, a disclaimer to the user within the application, in which it states that, the use of the app is for educational uses only, including features like contact form to consult with a doctor, patient history form, and language support for Arabic and English before the end of March 2026.
- I will meet with my supervisor, Dr. Ahmed Al-Adaileh, during every advisory meeting on Tuesdays, ensuring my project will go smoothly and asks for any advice that could assist me during my project.

2.1 Gantt Chart



Figure 2: Gantt chart

Note. The Gantt chart follows directly towards my objectives.

(<https://www.onlinegantt.com/#/gantt>)

3. Methodology

3.1 Software requirements

The project will be developed in Google Colab, using Python as its default programming language. The core deep learning frameworks for this project will be TensorFlow and Keras, which will build and train the Convolutional Neural Networks (CNN) models. Keras will provide the API to simplify the model's creation. I'll be using pre-trained CNN models such as ResNet50 and EfficientNet that are used for brain tumour classification. Data Augmentation techniques would assist by applying rotations to the images and reduce overfitting to improve the model's generalization. A GrabCut algorithm would be used for segmentation to improve classification accuracy. (Rao, et al., 2024).

Lastly, to connect the deep learning model with the mobile application, TensorFlow (via TensorFlow Lite) works very well for both Android and iOS. (Shaikh, 2021).

Learning how to create a deep learning model could be challenging as with no prior experience of using Google Colab, as well with the addition of creating a mobile app for the front-end. To achieve my goals, I'm taking both deep learning and mobile development modules to be successful in my final year project.

3.2 Hardware requirements

The hardware requirements used for this project is an HP Pavilion Gaming 15 laptop, which is running Windows 10 Home with version 22H2, a 64-bit operating system, x64-based processor, its built with a Ryzen 5 4600H with Radeon Graphics for the CPU @ 3.00 GHz, a NVIDIA GTX 1650 with 4GB of VRAM for the GPU, an 8GB (3200MHz) of RAM.

3.3 Development Methodology for the project

The Software development life cycle I plan on using is the Iterative model, as it starts the iterative process with simple implementations of a small set of software requirements, and iteratively enhances the evolving version until the complete system is implemented and ready to be deployed. (tutorialspoint, n.d.). I will develop my model iteratively, focusing on one interactive at a time.

In each iteration, I would train my model using the datasets and analyze the results to make sure if any necessary adjustments would be needed before proceeding to the next stage. In iteration, I would gather the requirements then design and developed the model and finally test and implement improvements towards the next build.

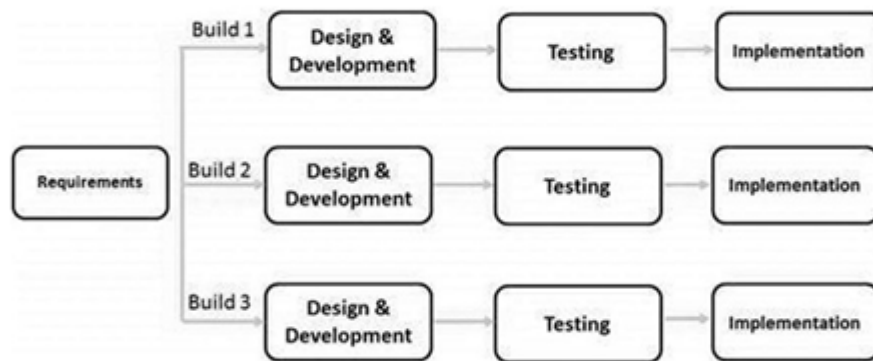


Figure 3: Iterative model

Note. Diagram of an iterative model by tutorialspoint. (https://www.tutorialspoint.com/sdlc/sdlc_iterative_model.htm)

Appendix B: Project Logbook

Date	Hours spent	description	Progress made	Problems encountered	Lesson learnt
21/9/2025	1 hour	Attended the week 0 seminar for final year project, met the new lecture for deep learning and mobile development.	Contacted with the new supervisor, Dr. Ahmed	Discussed ideas on my fyp and provided with information on what I can do better for it.	-
22/9/2025	3 mins	Send an email to Dr Ahmed to be my supervisor detailing my projects idea on "AI image detection system on brain cancer"	Responded saying that they'll be happy to be my supervisor	-	-
28/9/2025	10 mins	I talked to Dr Ahmed, and he suggested for me think more about different ideas and give examples from other students' projects. We plan to meet again on the	Help me to reflect more on personal problems that could be a successful project	-	I learnt a German phrase from my supervisor "everybody is just cooking with water" which means that your problems are similar to the people around you.

		30 th for the advisory meeting. I should come up with a real-life problem with my personal life and if I can't I'll work on my original idea.			
28/9/2025	50 mins	First week seminar had an introduction to final year project and deadlines for each part	-	-	You should Read the fyp guide during your preparation for the project
30/9/2025	30 mins	I talked to Dr Ahmed about what was my final decision. We talked on the best case to use the dataset for it and different features that could be added to it, and asked me to send an email to him on the title, explanation and features to add etc.	-	-	-
1/10/2025	3 mins	Send an email to Dr	Responded saying	-	-

		Ahmed regarding the title, description and features for my finalized project.	that he'll get back to me soon, which he did.		
5/10/2025	50 mins	Second week seminar was an introduction on how to write the project proposal and key points for it.	-	-	-
7/10/2025	12 mins	I talked with Dr Ahmed about the recent email and its follow-up, we talked about a new feature that could be added, which was contact information, where they can email with professionals, and to ask Dr Rowani for confirmation,	I was advised to talk with Dr Rowani, for final confirmation to which he approved, and I started to work on my proposal.	-	-
10/10/25	(1pm - 6pm) 5 hours	I started to work on the introduction and read articles on brain tumors.	I wrote down 300 words as of now, read 8 resources, to help me understand different topics well and paraphrased it	Difficulty in understanding a few research papers, as time was spent on re-reading and things were scraped	I know a little bit more about brain tumors and MRIs than before.

			on my words for the introduction	from what I wrote	
11/10/2025	(3:30pm - 7 pm) 3.5 hours	I resumed my previous work from yesterday, and read articles on deep learning when detecting brain tumors	I was able to write even better information.	Most of the content I covered from yesterday wasn't what I needed.	
12/10/2025	12:35-1:17pm, (42 mins)	Third week seminar, which conducted useful slides about methods for the final year project	-	-	-
12/10/2025	1:30pm – 2:00pm (30 mins)	After the seminar, I researched methodologies that I could use for my project,	I'll be using the Iterative model, as it fits best with my project.		
13/10/2025	12:30pm – 2:30pm (2 hours)	I wrote down the methodologies and objectives	-		
13/10/2025	9pm-11pm	I re-read my work and edited it	-	A few grammatical errors were found in	

	(2 hours)			my proposal.	
14/10/2025	1;25pm – 2:10pm (45 mins)	I had to delete a few sentences, to fix my structure and add a few additional points.	-	-	-
14/10/2025	3:30 pm – 4:00pm (30 mins)	I talked with Dr. Ahmed, we looked into my proposal, and he guided me to research on topics that would help me with my project.	-	-	He advised me to search on APIs and SDK, as it would help with the mobile app.
14/10/2025	9pm – 12pm (3 hours)	Wrote down the citations and reference list for the information I've written, wrote more for the objectives.			
17/10/2025	3:20pm – 5:40pm (2 hours and 20 mins)	Writing the conclusion for the proposal, rechecking all the citations, information added, a thorough checkup on everything I've written.	I found more ways to write my work better and added more information	Struggling with what is needed in my proposal	
19/10/2025	12:35pm – 1:15pm	Fourth seminar,	-	-	-

	(40 mins)	which was aimed towards the literature review on how to source good research papers for the project.			
19/10/2025	2:40pm – 3:40pm (1 hr)	I needed to add points on who my client and customer is and their benefits, my previous information on it, wasn't in detail.	I completed adding the beneficiaries and finished the proposal.		
20/10/2025	4:10 – 4:25pm (15 mins)	I talked with my supervisor, and he was pleased with my work, and we cleared the topic about discussing what kinds of APIs I could research on.	My proposal is up to date and it's ready for submission.	-	-
23/10/2025	-	I submitted my proposal via blackboard	-	-	-
26/10/2025	12:35 – 1:15	Week 5 seminar, this was aimed at the	-	-	-

	(40 mins)	requirements needed for the project, such as functional, non-functional and usability requirements.			
2/11/2025	12:35 – 1:25 (50 mins)	Week 6 seminar, this was aimed at learning about good methodologies and securities of the application.	-	-	-
9/11/2025	12:35 – 1:30 (55 mins)	Week 7 seminar, a summary guide to help us on how to write for our literature review.	-	-	-
10/11/2025	5:40pm – 7:20 pm (1hr and 40 mins)	I started to work on simple image classification model to differentiate between cats and dogs using CNN.	I followed a guide on YouTube, and it works well on PyCharm, the model was able to provide good accuracy.	However, there were some issues on its predictions between cats and dogs, it wasn't always accurate.	This was first time classifying images and predicting what kind of animal it is and I enjoyed learning it.
11/11/2025	3:00 – 4:00 (1 hr)	My supervisor provided us with a 1hr discussion in room 500, breaking down what he's	I noted down every important thing discussed and clarified my doubts with him in the	-	I was able to visualize my literature review more clearly now

		expecting from us for the literature review.	class.		
11/11/2025	4:00 – 5:00 (1 hr)	I discussed with Dr. Ahmed my progression of the fyp, where yesterday I started to work on dogs and cats' classification using CNN,	We discussed how the accuracy of the model could be improved by adding more layers and using a better dataset	-	The image I was using wasn't clear enough, and I should have double checked with the dataset.
18/11/2025	1:30 – 4:00 (2hr and 30 mins)	I continued working on my cats and dogs' model to better improve the model.	I've used new methods such as augmentation and dropouts as given to us during the workshops, and the model was doing well, images that I've used were clearer.	Although, the accuracy of the model didn't go higher than 73% in testing	I learned new insights to improve my model's accuracy using augmentation.
18/11/2025	7:30 – 8:30 (1 hr)	I started to practice a new CNN model, for X-rays detecting pneumonia and normal MRI scans.	This model performed better than the cats V dogs' dataset. I achieved an accuracy without augmentation of 79.5%,	Just in the beginning, when managing with file paths, and executing the code, I forget to change labels from cats v dogs'	Understanding and getting more comfortable with image classification models.

			with augmentation of 87.0% and with dropout of 82.2%	notebook.	
15/12/2025	7:30pm – 9:00pm	I started to work on the literature review by reading research paper on “ <i>Brain Tumor Detection Based on Deep Learning Approaches and Magnetic Resonance Imaging</i> ”	I finished reviewing my first paper and wrote on it.	-	-
16/12/2025	4:00pm – 6:30pm	I reviewed my first paper and started my research on the second paper, which was called “ <i>MRI-based brain tumor image detection using CNN based deep learning method</i> ”	I finished writing and reviewing my second paper for the literature review.	I haven’t encountered any problems whatsoever	I learned how CNNs are commonly used for MRI brain tumour detection.
17/12/2025	5:30pm – 7:00pm (1.5 hrs)	I continued reading <i>MRI-based brain tumour image</i>	I understood how preprocessing and data splitting affects	Some parts needed rereading to fully understand	I learned the importance of preprocessing in

		<i>detection using CNN based deep learning method and compared it with my own CNN experiments.</i>	model accuracy.		medical image classification.
18/12/2025	6:00pm – 7:30pm (1.5 hrs)	I started reading Paper 3, <i>ResNet50-based Deep Learning model for accurate brain tumor detection in MRI scans.</i>	I understood why ResNet50 is used as a transfer learning model.	Understanding the deeper architecture at first.	I learned how transfer learning improves accuracy and reduces training time.
19/12/2025	4:30pm – 6:00pm (1.5 hrs)	I continued reviewing the ResNet50 paper and linked it to my planned advanced model.	-	-	I learned how pre-trained models help with limited datasets.
20/12/2025	7:00pm – 8:30pm (1.5 hrs)	I started reading Paper 4, <i>Enhancing Brain Tumor Detection Using a CNN-Powered Android Application.</i> And looked into the	I understood how CNN models can be deployed into mobile applications.	Understanding the Android implementation	I learned how deep learning models can be used in real-world mobile apps.

		methodology as well			
23/12/2025	5:00pm – 6:30pm (1.5 hrs)	I finished reviewing the Android-based brain tumour detection paper and summarized it for my literature review.	I completed the final paper summary for the literature review	-	I learned how CNN models can be optimized for mobile deployment.
26/12/2025	2:00pm – 5pm (3 hrs)	I started to write the last section of the report, which was the methodology	I completed a clear draft explaining the project methodology and tools used.	Structuring the section clearly first.	I was able to explain my development well later
29/12/2025	4pm – 5pm (1hr)	I reviewed my report one last time, focusing on grammar and overall structure.	I fixed grammar mistakes and made sure the report was well-structured and clear.	-	-
4/01/2026	-	I submitted my work on the submission date, even though the work was already done	-	-	-
27/01/2026	2:00pm – 2:30pm (30 mins)	I talked with my supervisor about my literature review, to discuss any fixes or to better re-structure for my report.	I have yet to fix my lit review again. But ill work on the implementation of my work first.	-	-

30/01/2026	1:00pm – 5:30pm (4.5 hours)	I began writing the code in google colab, and I started with the VGG-16 model for the time being.	After running the code for 30 mins, I have received a testing accuracy of 87%.	A lot of adjustments are still needed to reach an accuracy of at least 95%+	-
01/2/2026	7pm – 9pm (2 hours)	I fine-tuned my model and received much better results.	I received a testing accuracy of 95.58% (96). A huge improvement from the previous model	After trial and error, my confusion matrix diagram wasn't displayed, although it's fixed as well.	-
02/2/2026	5pm -7pm	I worked on my model again, and 2 classes weren't performing well as the rest	I couldn't solve it, and I also felt ill.	-	-
09/2/2026	12pm – 12:50pm	Week 3 of semester two's seminar was presented to us, with helpful approaches to our final year project,			
10/2/2026	11:30am – 12pm	I talked with my supervisor about possibly using new data sets of MRI images, that aren't	My supervisor is happy to help with my idea, but I need to get more clarification from	-	-

		publicly available and what approach should I do to get an ethical approval form.	other professors.		
11/02/2026	8pm – 11pm	I started to get back to work and went to fix my previous problem.	I achieved an accuracy of 98%, and with just minor issues between glioma and meningioma, they just have a small confusion. Rest everything else is perfect	I need to fix 2 classes, to make sure they're fully accurate for the model.	-
13/02/2026	1pm – 6pm	I kept trying to fix the previous class issue, but I'll keep as it is for now and work on ResNet50 for now.	I use the same approach as vgg16 but unfroze the last 30 layers.	The model initially took so long to run over 1hr 30 mins, for only 7 epochs, cos I forgot to change the runtime type; after changing it, it was done in less than 40 mins.	Keep in mind if the runtime is GPU and not my local machine's CPU, cos my machine CPU isn't the best.
14/02/2026	11:30am – 5pm	I began working on a 3 rd model, which was EfficientNetB0. In hope maybe a	As of now, VGG16 performed the best accuracy. Resnet50 and EfficientNetB0	-	-

		better model will outperform the first 2 models.	were alright, but I added 15 epochs additional to the vgg16, and it performed an accuracy of 98.77%.		
14/02/2026	11:30am – 5pm	After reaching an amazing accuracy, I began working on a basic flask app in vscode, to test the model.			
14/02/2026	9pm – 1am	I started to work on a rough flask app for the models (I don't plan on using it, but just for experimenting)	It took me a lot of time to complete it, but I believe it's working perfectly now	I had to change the python versions, change the keras file to h5, add the code of the model, it was draining, but the flask app works	-
17/02/2026	2pm – 2:30pm	I talked with my supervisor, and we discussed problems and issues with my recent work, and I must work on it again later this week.	I'm glad of my progress but I'll need to do some re-working on certain parts.	Issues with the pre-processing stage and code presented.	I got a better insight of my work, thanks to the support of my supervisor.

20/02/2026	2pm - 5pm	I changed my model, to fit with the description given to me based on my feedback from my supervisor.	I achieved an accuracy of 98%, without any faults in my work.	-	-
24/02/2026	11:40am – 12:30pm	I talked with my supervisor, and he was happy with my progress, as I did exactly what he wanted me to do.	I reassured myself that my model is successfully perfect and I can work to the next stage. He advised me to research on APIs, so I could add connections between my models and the application of my choice.	-	-
1/03/2026	1pm – 4pm	I began continuing where I left off and worked on the APIs.	I couldn't get any progress, rather lots of issues and confusion	I don't think I'll be able to do one with fast API, I'll try with android studio or flask app.	-
3/03/2026	9pm – 3am	I tried my best to work with TensorFlow lite in android studio, but it was difficult to	Instead, flask apps work a lot easier and have less demand on my laptop, as they're very fast.	Running flask was a problem, cause of different versions when dealing with TensorFlow, keras	

		manage, on how laggy it is.		etc.	
5/03/2026	11:30 am - 12:00 pm	I spoke with my supervisor about my recent changes on what application I'll be working on to which he's happy with and showed him my prediction model in flask, which is perfect as well.	My supervisor is happy with my work thus so far, all I have to do is finish the flask app, as his remark was to improve the UI of it.	-	-
7/03/2026	5pm – 7pm	I worked on a rough sketch for the screens of the flask app.	I was able to complete this task, as I already had a general idea on how I was going to plan it out.	-	-
10/03/2026	6pm – 9pm	I did some screens for the flask app, needs improvement with better css and images	I went through different HTMLs, to fix issues and added better colors to match with current medical apps.	Certain implementations for the flask app, I had to learn the concepts of it.	

13/03/2026	1-4pm	Added translation tools like Arabic, plan to do Urdu as well. Added better buttons	I converted all the English letters to Arabic using google translate, and its not hard coded, I used a reference for it in translations.py	-	-
15/03/2026	6pm – 10pm	I completed the final design changes, bugs and styling, by the 15 th of March. I'm waiting to show my work to my supervisor, after the spring break.	I went through all the code files to make sure everything was working smoothly and efficiently.	I only came up with one issue, and it was deploying, I was unsuccessful in doing it, I tried every tool, website, but it wasn't working.	-
21/03/2026	2pm – 4pm	After completing the practical work, I began writing the report, with first creating better wireframes for the flask app.	I finished the wireframes, but plan to show this to my supervisor for approval.	-	-
26/03/2026	2pm – 7pm	I finally started working on the requirements, designs and planning chapter	I wrote use cases, user stories, use case diagram, functional and non-functional requirements	-	-

27/03/2026	2pm – 6pm	I continue working on the same chapter and re-reading what I wrote yesterday.	I wrote the HTA analysis, textual use cases, sequence diagram and high-level system architecture.	-	-
29/03/2026	5pm – 9pm	I still had to write a lot more on chapter 4, and kept fixing the tone of my work	I was able to design a few diagrams for the report	Nevertheless, I still faced a few roadblocks on completing a few sections	
01/04/2026	4pm-4:30pm	I spoke with my supervisor about my work and asked for any changes needed	He suggested a few ideas on changing the primary actor, as a patient using the system won't be appropriate and features such as a login system		

03/04/2026	1pm – 8pm	During the start of April, I began working a lot more than usual, to make sure I'm able to finish the report on time and have time to do other assignments on the side	I was able to get some progress done for chapter 4	I was still not able to finish chapter 4; I was getting frustrated.	
5/04/2026	2pm – 8pm	I faced an issue with flaks, as I tried testing the other keras file and faced with a lot of errors,	I was only able to complete chapters of chapter 4	I couldn't solve the flask issue.	-

		to which I started the report as well			
7/04/2026	2pm - 7pm	I skipped a few sections of chapter 4 to do them later as I wasn't sure what to add, and started on chapter 5, and I fixed the issue for Flask	I started a few sections for chapter 5, found suitable papers, and the issue in flask was resolved by fixing the versions for flask and the keras file	-	-
09/04/2026	11:30am – 12pm	I spoke with my supervisor about my final works progress and concluded as my last session.	I just wanted to clear my concerns for my deep learning modes as I was doubting my accuracy that I have achieved.	- -	-
11/04/2026	1pm – 8pm	I worked again on the report and during the days in between I was working on mobile development	I completed chapter 5 by finishing the reaming sections and chapter 6	-	-
15/04/2026	3pm – 8pm	I first started on mobile to make sure I have progressed on it as well, afterwards I worked on the final year project report	I was able to finish chapter 7	-	-

16/04/2026	3pm – 7pm	I researched suitable papers for comparing models, to compare my results with others.	I was able to complete this within a few hours and planned on finishing on other sections	I made slow progress today; I didn't complete as much work as I did yesterday.	
18/04/2026	2pm – 8pm	I completed working on SQM, and I also continued working on my final year project	I was able to finish chapter 8, introduction and design the Gantt chart.	-	-
19/04/2026	1:30pm – 2:15pm	I spoke with my supervisor to show my overall structure of my report and asked for feedback	Most of my sections were well written	Although some of them needs improvement	-
19/04/2026	3pm – 7pm	I re-edited some of my sections to fit well with the feedback given to me and overall to complete my work	I just read my work	I wasn't able to work on my report during this day, due to feeling overwhelmed	

20/4/2026	12pm – 9pm	I did work on fixing my list of tables, figures, structure of my report, adding more content to chap 4, chap 6 for testing,	I was able to complete most of my report. I finished almost everything. I just need to read the whole thing. Which I will be done by tomorrow.	-	-
21/04/2026	2pm - 8pm	I worked on checking all my code is correct, commented correctly, and review if i missed on anything important in the report	I was able to complete all tasks successfully.	-	
22/04/2026	2pm - 6pm	My goal is to read and submit the report today	I have officially submitted my report.	-	Over the last seven months, I have learned a lot, not just about the report, but also about my ability to take on an immense task and complete it on time.

